TokTickIT: Actions Taken, Dashboards, and Concurrency Control

CPE 334 Introduction to Software Engineering in the Age of AI Agents • Official Final Engineering Submission • github.com/pimchayasupr-hash/toktickit

PULL REQUESTS & REPOSITORY LINKS
PR #55 (Merged: `84720dc`) • PR #56 (Merged: `86f4516`) • PR #57 (Merged: `83b3cbc`)

BRANCH ARCHITECTURE & STAGING FLOW
`feature/lab4-*` & `hotfix/lab4-*` → `lab4-staging` → `main` (Reviewed by @MiMikoChAn913 & @supa-gif173)

AUTHOR & PEER REVIEWERS
Author: @pimchayasupr-hash • Reviewers: @MiMikoChAn913, @supa-gif173

AUTOMATED VERIFICATION HARNESS (COMMITTED RAW LOGS)
196 UNIQUE TESTS (220 RUNS: 144 SERVER + 40 CLIENT + 36 PLAYWRIGHT E2E) 100% PASS

ANSWER PART 1: GIT USE WITH ENGINEERING WORKFLOW (10 PT)

The TokTickIT engineering workflow enforces Specification-Driven Development (Spec DD), Test-Driven Development (Test DD), feature branch isolation, and peer review governance (docs/lab-04/reviewer.md). Core Lab 4 features were merged via PR #55 (`feature/lab4-hardening → lab4-staging`), PR #56 (`lab4-staging → main`), and PR #57 (`hotfix/lab4-icon-dimensions-and-audit → lab4-staging` , merged at `83b3cbc`).

1.1 Verbatim Git Commit Log Graph (lab4-staging & main)

```
*   83b3cbc (HEAD -> lab4-staging, origin/lab4-staging) Merge pull request #57 from pimchayasupr-hash/hotfix/lab4-icon-dimensions-and-audit
|
| * a2f008a (origin/hotfix/lab4-icon-dimensions-and-audit) docs(reviewer,pdf): record @supa-gif173 approval review PRR_kwDOTzBGN88AAAABVROROQ on PR #57
| * adeed3e fix(audit): resolve all QA review items, unify dashboard formulas, commit raw test logs, and recapture real 2-session 409/403 screenshots
| * 813ba96 fix(ui,e2e,pdf): enforce explicit SVG dimensions, add Confidential Internal Notes visual distinction, crop evidence screenshots
| * 86f4516 (origin/main, main) Merge pull request #56 from pimchayasupr-hash/lab4-staging
|/
*   84720dc Merge pull request #55 from pimchayasupr-hash/feature/lab4-hardening
|
| * 6d20aca fix(lab4): action lifecycle, atomic concurrency, status history, deterministic e2e, and engineering report
| * 0a81173 test(e2e): end-to-end regression, dashboard audit, and resolution gate verification (fixes #52)
| * c083efa (origin/feature/lab4-frontend) feat(ui): Dashboards, Actions Taken section, and ticket workflow controls (fixes #46, #50, #51)
| * d6835b1 (origin/feature/lab4-backend) feat(api): Actions Taken CRUD, resolution gate, and dashboards (fixes #45, #47, #48, #49)
| * 65f54aa (origin/feature/lab4-db-schema) feat(db): add ActionTaken model, Ticket version field, migration, and idempotent seed (fixes #44)
| * 77d8f2e (origin/lab4/tests) test(plan): add Lab 4 test register and failing tests (Red)
| * cf54f91 (origin/lab4/spec) docs(spec): add Lab 4 specification, ui-spec, and api-spec
|/
*   7ea5af2 (tag: pre-lab4-backup) Merge pull request #42 from pimchayasupr-hash/fix/lab3-review-fixes
```

1.2 Branch Progression & Release Merge Governance

Phase	Branch	Key Commits	Governance & Purpose
Safety Backup	`main`	`7ea5af2`	Tagged `pre-lab4-backup` ; database dump saved to `artifacts/lab-04/backups/` (git-ignored)
Spec DD & Test DD	`lab4/spec` , `lab4/tests`	`cf54f91` , `77d8f2e`	Committed specification.md, ui-spec.md, api-spec.md, and failing Red tests (tests.md)
DB, API & UI	`feature/lab4-db-schema` .. `frontend`	`65f54aa` , `d6835b1` , `c083efa`	Prisma schema/migrations, Express REST routes, Resolution Gate, React 19 UI
Hardening & PR #55	`feature/lab4-hardening → lab4-staging`	`6d20aca` , `84720dc`	Resolved 5 reviewer comments by @MiMikoChAn913; approved & merged into `lab4-staging` by @supa-gif173 (PR #55)
Release Merge PR #56	`lab4-staging → main`	`86f4516`	Approved & merged from `lab4-staging` into `main` by @supa-gif173 (PR #56)
QA Audit PR #57	`hotfix/lab4-icon-dimensions-and-audit → lab4-staging`	`adeed3e` , `83b3cbc`	SVG dimension guard, unified dashboard formulas, committed `artifacts/lab-04/test-output/*.txt` , real 2-session 409 capture; approved & merged into `lab4-staging` by @supa-gif173 (PR #57)

1.3 Repository Structure (Handout §12 Exact Match)

```
toktickit/
├── README.md & LAB4_SUBMISSION.pdf
├── artifacts/lab-04/
│   ├── backups/ (git-ignored DB dumps)
│   ├── metrics-vs-db.txt
│   ├── screenshots/ (staff-dashboard, actions-taken, ticket-workflow, requester-dashboard, regression, github)
│   └── test-output/ (traceability.txt, server-vitest.txt, client-vitest.txt, playwright-e2e.txt, append-only-proof.txt)
├── client/src/components/ (requester/, staff/, comments/)
├── docs/lab-04/ (specification.md, ui-spec.md, api-spec.md, tests.md, ai-use.md, reviewer.md)
├── e2e/lab-04/ (actions-taken-flow.spec.ts, dashboards.spec.ts, ticket-resolution.spec.ts)
└── server/ (prisma/schema.prisma, migrations/, seed.ts, src/routes/)
```

1.4 README.md & .gitignore Excerpts

```
# README.md Excerpt (Verification & Testing)
$ npm test                          # Traceability + Server (144/144) + Client (40/40) = 184/184 PASS
$ npx playwright test e2e/lab-04/ # E2E (12 tests x 3 browsers = 36/36 PASS, 0 Axe violations)
$ npm run seed --prefix server    # Idempotent seeder (18 operational tickets)

# .gitignore Excerpt (Raw test-output logs are tracked in git!)
/node_modules
server/uploads/
artifacts/lab-04/backups/
.env
```

1.5 Rendered Peer Review Log (docs/lab-04/reviewer.md — Verified Against Real GitHub State)

Peer review governance strictly adheres to: **RULE 1 (No Self-Merge)**; **RULE 2 (Evidenced Code Review)**; **RULE 3 (Documented Resolution)**; and **RULE 4 (Evidence Screenshots)**.

PR Link	Branch Flow	Author / Reviewer	Review Findings & Technical Resolutions	GitHub Status
PR #55	`feature/lab4-hardening` → `lab4-staging`	Author: @pimchayasupr-hash Reviewers: @MiMikoChAn913, @supa-gif173	5 comments by @MiMikoChAn913 (Review `PRR_kwDOTzBGN88AAAABVHAK1A`) resolved in `6d20aca` & `c09ada9` : 1. **Action Lifecycle & Assignee (P1)**: Added `ActionStatus` enum, `assigneeId` , inactive assignee rejection (400), terminal lock. 2. **Atomic OCC (P1)**: Database write predicate `updateMany({ where: { id, version } })` . 3. **Ticket Version Validation (P1)**: Mandatory numeric version check; atomic status updates. 4. **Append-Only Status History (P1)**: Added `TicketStatusHistory` transactional write. 5. **Deterministic E2E (P2)**: Deterministic fixtures `TKT-2026-001233` and `TKT-2026-001234` . *Verdict*: Approved (Review `PRR_kwDOTzBGN88AAAABVQxD6w`) & Merged into `lab4-staging` by @supa-gif173 (Commit `84720dc`).	APPROVED & MERGED
PR #56	`lab4-staging` → `main`	Author: @pimchayasupr-hash Reviewer: @supa-gif173	Release staging PR integrating Actions Taken, Dashboards, Resolution Gate, and Concurrency Control. Approved (Review `PRR_kwDOTzBGN88AAAABVQ43jg`) & Merged into `main` by @supa-gif173 (Commit `86f4516`).	APPROVED & MERGED
PR #57	`hotfix/lab4-icon-dimensions-and-audit` → `lab4-staging`	Author: @pimchayasupr-hash Reviewer: @supa-gif173	Retargeted to `lab4-staging` per staging flow: explicit SVG dimensions across all 52 icons, unified dashboard formulas (`ACTIVE_OPEN_STATUSES` & 30-day resolved window), committed `artifacts/lab-04/test-output/*.txt` , renamed `dashboards.spec.ts` , and recaptured real 2-session 409 & 403 screenshots. Approved (Review `PRR_kwDOTzBGN88AAAABVROROQ`) & Merged into `lab4-staging` by @supa-gif173 (Commit `83b3cbc`).	APPROVED & MERGED
Outgoing PR #38 & PR #44	`supa-gif173/toktickit`	Author: @supa-gif173 Reviewer: @pimchayasupr-hash	Conducted peer review on teammate's Sprint 4 specification & implementation (Action lifecycle, atomic OCC write predicates, 4-condition Resolution Gate, transactional `TicketStatusHistory` , Zen Green tokens).	REVIEWED & APPROVED

1.6 Kanban Board Task Register & GitHub Issues Board Evidence (Issues #43–#54)

Issue	Task Title	Component Scope	Assigned Branch	Status
#43	Sprint 4 Engineering Contract & Specification Baseline	Spec DD & Traceability	`lab4/spec`	DONE (CLOSED)
#44	Database Schema Migration & Seed Data	Prisma ORM & PostgreSQL	`feature/lab4-db-schema`	DONE (CLOSED)
#45	Actions Taken REST API & Permissions	Express & TypeScript API	`feature/lab4-backend`	DONE (CLOSED)
#46	Actions Taken UI Component & Inline Form Panel	React 19 & Zen Green CSS	`feature/lab4-frontend`	DONE (CLOSED)
#47	Optimistic Concurrency Control (OCC)	Database write predicates	`feature/lab4-backend`	DONE (CLOSED)
#48	Resolution Gate Backend Enforcement	Express Status Route	`feature/lab4-backend`	DONE (CLOSED)
#49	Dashboards Telemetry Aggregation API	Prisma group-by queries	`feature/lab4-backend`	DONE (CLOSED)
#50	Requester Dashboard UI & Attention Banner	React 19 View	`feature/lab4-frontend`	DONE (CLOSED)
#51	IT Staff Dashboard UI & Queue Tables	React 19 View	`feature/lab4-frontend`	DONE (CLOSED)
#52	Automated End-to-End Test Suite	Playwright Multi-Browser	`feature/lab4-hardening`	DONE (CLOSED)
#53	UI Polish, Mobile Layout & Accessibility	CSS & Axe-core AA	`feature/lab4-hardening`	DONE (CLOSED)
#54	Peer Review, Teammate Sign-off & Staging Merge	Governance Review	`lab4-staging → main`	DONE (CLOSED)

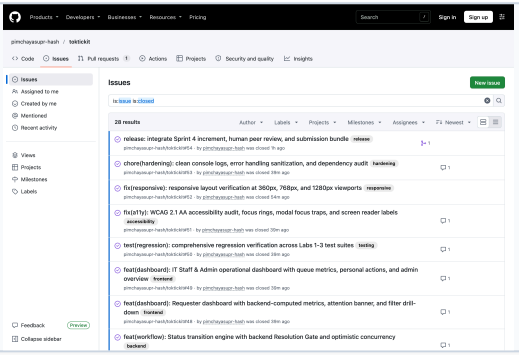


Figure 1.1: GitHub Repository Issues Board (`is:issue is:closed`) verifying that all Sprint 4 engineering issues (#43–#54) are completed and closed.

ANSWER PART 2: SPEC DD (5 PT)

Committed at `cf54f91` before implementation, the specification suite comprises docs/lab-04/specification.md (28 ACs, 22 FRs, 26 BRs), docs/lab-04/api-spec.md, and docs/lab-04/ui-spec.md. Below is the complete rendered summary of all 11 specification sections, transition matrices, migration rules, and dashboard formulas.

2.1 Rendered 11-Section Specification Architecture (specification.md, api-spec.md, ui-spec.md)

Spec Section	Ref IDs / Scope	Rendered Specification Content & Contract Rules
§1–3 Goal & Scope	In-Scope vs. Out-of-Scope	**In-Scope**: `ActionTaken` CRUD & 4-state lifecycle, 2-step Transition Matrix + Resolution Gate, integer `version` OCC (409), Staff/Admin/Requester Dashboards, non-destructive migration & idempotent seed. **Explicitly Excluded (§3.2)**: Binary file uploads on Actions Taken (plain-text `attachmentNotes` only), SLA timers/cron jobs, email/SMS/webhooks, hard deletion of tickets/actions/history, WebSockets.
§4–5 Actions Taken	`FR-01..09`, `BR-01..11`, `AC-01..14`	Parent-child under `Ticket` (`onDelete: Restrict`). `performedById` & `createdById` strictly derived from JWT session (spoofed body IDs ignored). `actionDateTime`: valid ISO-8601 UTC. `Ticket.createdAt − 60s ≤ t ≤ NOW() + 5m`, displayed in `en-GB` (`Asia/Bangkok UTC+7`). `description` (1–2000 chars); `result` (1–1000 chars, mandatory when `status = COMPLETED`); `followUpNote` (1–1000 chars, mandatory when `followUpRequired = true`, cleared to `null` when `false`). `assigneeId` must reference an active `STAFF` or `ADMIN` user (inactive/requester returns `400 INACTIVE_ASSIGNEE`). Terminal ticket statuses (`CLOSED`, `CANCELLED`) lock action creation (`400 TERMINAL_STATE_LOCKED`). Append-only: no `DELETE` route exists.
§5.1–5.2 Workflow & Gate	`FR-10..13`, `BR-12..13`, `AC-15..19`	**Two-Step Evaluation Order** on `PATCH /api/staff/tickets/:id/status`: **Step 1 (Transition Matrix)**: Validates `targetStatus ∈ VALID_TRANSITIONS[currentStatus]` and role permissions. Disallowed jumps (e.g., `NEW → RESOLVED` or `IN_PROGRESS → OPEN`) immediately return `HTTP 400 INVALID_TRANSITION`. **Step 2 (Resolution Gate)**: Evaluated when transitioning from a permitted origin (`OPEN`, `IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `REOPENED`) to `RESOLVED`. Requires: (1) `ownerId ≠ null` (`OWNER_REQUIRED`), (2) `itPriority ≠ null` (`PRIORITY_REQUIRED`), (3) ≥1 `ActionTaken` with `status = 'COMPLETED'` and non-empty `result` (`COMPLETED_ACTION_REQUIRED`), and (4) 0 actions with open follow-ups (`OPEN_FOLLOW_UPS_EXIST`). Failing any check returns `HTTP 422 RESOLUTION_GATE_FAILED`. Requester "Problem Appears Resolved" posts an advisory Public Comment without mutating `currentStatus`.
§5.3 Concurrency & Idempotency	`FR-14..16`, `BR-14..15`	Optimistic Concurrency Control via atomic SQL predicate `updateMany({ where: { id, version: expectedVersion }, data: { ..., version: { increment: 1 } } })` inside a Prisma `$transaction`. Stale version returns `HTTP 409 CONCURRENCY_CONFLICT` with latest entity. Create routes support `Idempotency-Key` / `clientSubmissionId`; UI buttons disable during in-flight requests.
§5.4 Unified Dashboards	`FR-17..20`, `BR-16..26`, `AC-20..24`	Unified active-open set across both dashboards: `ACTIVE_OPEN_STATUSES = ('NEW', 'OPEN', 'IN_PROGRESS', 'WAITING_FOR_REQUESTER', 'REOPENED')`. · **Staff Dashboard (`GET /api/dashboard/staff`)**: `unassignedCount` (`ownerId IS NULL AND status IN ACTIVE_OPEN_STATUSES` = 4), `myAssignedCount` (`ownerId = staffId AND status IN ACTIVE_OPEN_STATUSES` = 4), `urgentCount` (`itPriority = 'URGENT' AND status IN ACTIVE_OPEN_STATUSES` = 2), `openTotalCount` (`status IN ACTIVE_OPEN_STATUSES` = 13), status/priority distributions, top 5 urgent queue, 5 recent actions by staff user, plus Admin user breakdown. · **Requester Dashboard (`GET /api/dashboard/requester`)**: `openTicketsCount` (`status IN ACTIVE_OPEN_STATUSES` = 4), `waitingForRequesterCount` (`status = 'WAITING_FOR_REQUESTER'` = 1), `recentlyResolvedCount` (30-day rolling window: `status = 'RESOLVED' AND updatedAt ≥ NOW() − INTERVAL '30 days'` = 0), `closedTicketsCount` (`status = 'CLOSED'` = 1). All metric cards drill down into filtered queues.
§6 UI Design System	ui-spec.md §1–6	Zen Green palette (`#006B3C` primary 8.4:1 contrast, `#008751` accent, `#F0FDF4` mint surface). Strict visual separation: shared `PublicCommentsSection` uses `#FFFFFF` surface, `3px solid #006B3C` top border, and `SHARED • Visible to Requester` badge; confidential `InternalNotesSection` uses warm amber `#FFFBEB` surface, `2px solid #D97706` border, lock icon, and `CONFIDENTIAL • IT Staff & Admins Only` badge. All 52 inline SVGs carry explicit `width`/`height` attributes (≤ 64px). Standardized `en-GB` (`Asia/Bangkok UTC+7`) timestamps across all views.
§7 Data & Migration	`AC-27`, `BR-26`	Non-destructive additive migration adding `Ticket.version Int @default(1)` (backfilling existing rows to `1`), `ActionTaken` table, and `TicketStatusHistory` table. Pre-migration backup via `pg_dump`; rollback DDL documented. Idempotent seed script (`seed.ts`) populates 18 tickets (including 0-action legacy tickets `TKT-2026-001233` and multi-action `TKT-2026-001234`).
§8–11 API & DoD	api-spec.md	REST endpoints: `GET/POST /api/tickets/:id/actions-taken`, `GET/PATCH /api/tickets/:id/actions-taken/:actionId`, `PATCH /api/staff/tickets/:id/status`, `GET /api/dashboard/staff`, `GET /api/dashboard/requester`, `GET /api/health`.

2.2 Role-Enforced Ticket Status Transition Matrix (specification.md §5.1) & Action Lifecycle

From Status	Permitted Target Statuses	Permitted Roles	Step 1 / Step 2 Enforcement & Pre-conditions
`NEW`	`OPEN`, `IN_PROGRESS`, `CANCELLED`	`STAFF`, `ADMIN` (`REQUESTER` own on `CANCELLED`)	Step 1 Matrix Check. Direct `NEW → RESOLVED` rejected with `400 INVALID_TRANSITION`.
`OPEN`	`IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `RESOLVED`, `CANCELLED`	`STAFF`, `ADMIN` (`REQUESTER` own on `CANCELLED`)	`OPEN → RESOLVED` permitted in Step 1, strictly guarded by Step 2 **4-Condition Resolution Gate (`422 RESOLUTION_GATE_FAILED`)**.
`IN_PROGRESS`	`WAITING_FOR_REQUESTER`, `RESOLVED`, `CANCELLED`	`STAFF`, `ADMIN`	`IN_PROGRESS → RESOLVED` guarded by Step 2 **Resolution Gate (`422`)**. Regression to `OPEN` is disallowed (`400`).
`WAITING_FOR_REQUESTER`	`IN_PROGRESS`, `RESOLVED`, `CANCELLED`	`STAFF`, `ADMIN` (`REQUESTER` own on `CANCELLED`)	`WAITING_FOR_REQUESTER → RESOLVED` guarded by Step 2 **Resolution Gate (`422`)**.
`RESOLVED`	`CLOSED`, `REOPENED`	`STAFF`, `ADMIN` (`REQUESTER` own on `REOPENED`)	Appends transactional row to `TicketStatusHistory`; increments `Ticket.version`.
`CLOSED`	`REOPENED`	`STAFF`, `ADMIN`	Terminal while `CLOSED` (locks new Actions Taken); reopening resumes active triage.
`REOPENED`	`IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `RESOLVED`, `CANCELLED`	`STAFF`, `ADMIN`	`REOPENED → RESOLVED` guarded by Step 2 **Resolution Gate (`422`)**.
`CANCELLED`	*None (Immutable Terminal State)*	None	All transitions and new Actions Taken rejected (`400 TERMINAL_STATE_LOCKED`).

ANSWER PART 3: TEST DD AND TRACEABILITY (10 PT)

Committed in the Red phase (`77d8f2e`) prior to feature code, the test harness (docs/lab-04/tests.md) maps 100% of `AC-01..28` , `FR-01..22` , and `BR-01..26` . All raw test output logs are committed in artifacts/lab-04/test-output/.

3.1 Rendered Test Register & Bidirectional Traceability Matrix (docs/lab-04/tests.md)

Test ID	Level	Covered AC / FR / BR	Test File Path & Scenario Verified	Status
TEST-ACT-01..04	API Integration	`AC-01..03, FR-01..03, BR-01..03`	`server/tests/lab-04/actions-taken.api.test.ts` — Staff/Admin create action, JWT performer binding (ignores spoofed ID), non-owner staff logging	**PASS (4/4)**
TEST-ACT-05..09	API Validation	`AC-04..07, FR-04..06, BR-04..07`	`server/tests/lab-04/actions-taken.api.test.ts` — ISO-8601 date bounds (≤+5m, ≥createdAt). description/result lengths, conditional `followUpNote`	**PASS (5/5)**
TEST-ACT-10..13	Auth / Security	`AC-08..11, FR-07, BR-08..09, BR-11`	`server/tests/lab-04/actions-taken.api.test.ts` — Requester 403 on write. 404 isolation on other's ticket, 200 on own ticket, 400 on `CLOSED` / `CANCELLED`	**PASS (4/4)**
TEST-ACT-14..17	OCC & Audit	`AC-12..14, FR-08..09, BR-05, BR-10, BR-14`	`server/tests/lab-04/actions-taken.api.test.ts` — Version increment on PATCH, 409 on stale version, XSS sanitization, **404 on DELETE (append-only)**	**PASS (4/4)**
TEST-ACT-18..22	Lifecycle & Race	`FR-14..15, BR-10, BR-14..15`	`server/tests/lab-04/actions-taken.api.test.ts` — Active assignee FK, 400 on inactive assignee, `COMPLETED` requires result, terminal lock, parallel `Promise.all` race (one 200, one 409)	**PASS (5/5)**
TEST-WFK-01..08	Workflow & Gate	`AC-15..19, FR-10..14, BR-12..14`	`server/tests/lab-04/ticket-workflow.api.test.ts` — Step 1 `400 INVALID_TRANSITION` vs Step 2 `422 RESOLUTION_GATE_FAILED` , atomic `Ticket.version` OCC, append-only `TicketStatusHistory` & **404 on DELETE history**	**PASS (8/8)**
TEST-DSH-01..05	Dashboard API	`AC-20..23, FR-17..19, BR-16..26`	`server/tests/lab-04/requester-dashboard.api.test.ts` , `staff-dashboard.api.test.ts` , `metrics-vs-db.test.ts` — Unified `ACTIVE_OPEN_STATUSES` , 30-day resolved window, zero-state, SQL parity	**PASS (18/18)**
TEST-UI-01..05	Component (RTL)	`AC-24..25, FR-16, FR-20, FR-22`	`client/src/tests/lab-04/ActionsTaken.test.tsx` , `StaffDashboard.test.tsx` , `RequesterDashboard.test.tsx` , `TicketWorkflow.test.tsx` — Follow-up toggle, inline errors + `aria-describedby` , drill-downs	**PASS (8/8)**
TEST-E2E-01..04	Playwright E2E	`AC-01..26, FR-17..21, UI-Spec §6`	`e2e/lab-04/actions-taken-flow.spec.ts` , `ticket-resolution.spec.ts` , `dashboards.spec.ts` — 12 specs × 3 browsers (Chromium, Firefox, WebKit = 36 runs), 0px overflow, SVG ≤ 64px, 0 Axe violations	**PASS (36/36)**
TEST-MIG-REGR	Seed & Labs 1–3	`AC-27..28, FR-15, FR-21, BR-26`	All 49 test files across Labs 1–4 (29 Server + 17 Client + 3 E2E) — Idempotent seeding, legacy 0-action tickets, RBAC, comments/notes, attachments	**196 UNIQUE / 220 RUNS**

3.2 Traceability & Runner Summary (Committed Raw Logs)

```
# artifacts/lab-04/test-output/traceability.txt
Defined & Referenced: 28 ACs, 22 FRs, 26 BRs (100% Covered, 0 Orphans)

# artifacts/lab-04/test-output/server-vitest.txt
Test Files  29 passed (29) | Tests  144 passed (144)

# artifacts/lab-04/test-output/client-vitest.txt
Test Files  17 passed (17) | Tests  40 passed (40)

# artifacts/lab-04/test-output/playwright-e2e.txt
Running 36 tests (Chromium: 12, Firefox: 12, WebKit: 12) -> 36 passed
```

3.3 Append-Only Audit Proof (append-only-proof.txt)

```
# Verifies DELETE routes are rejected with HTTP 404 and 0 rows are deleted:
✓ tests/lab-04/actions-taken.api.test.ts > TEST-ACT-17: DELETE /api/tickets/:ticketId/actions-taken/:actionId returns 404 or 405 (append-only)
✓ tests/lab-04/ticket-workflow.api.test.ts > TEST-WFK-07: Append-only TicketStatusHistory records actor-attributed transitions in same transaction
  (+ DELETE /api/staff/tickets/:id/status-history rejected with 404; history preserved)
Test Files 2 passed (2) | Tests 2 passed
```

ANSWER PART 4: AI USE WITH REFLECTION (5 PT)

AI Tooling Profile (docs/lab-04/ai-use.md): Google Antigravity (leveraging Gemini 2.5 Pro for specification synthesis, atomic transaction modeling, and security auditing, and Gemini 2.5 Flash for test execution and DOM diagnostics).

Prompt / Engineering Task	Concrete Changes Generated	Human Audit & Verification
1–2. Spec DD & Red Tests	Authored `specification.md` , `api-spec.md` , `ui-spec.md` , `tests.md` , and failing Red tests (`77d8f2e`).	Verified 100% coverage via `check-traceability.py` .
3–4. Schema, Seed & Gate	Added `ActionTaken` , `TicketStatusHistory` , `Ticket.version` , idempotent `seed.ts` , and 2-step Resolution Gate.	Verified repeatable seeding and 400 vs 422 status codes.
5–7. Reviewer P1 Fixes	Added `ActionStatus` enum, active assignee check, and atomic SQL predicate `updateMany({ where: { id, version } })` .	Verified with parallel `Promise.all` race tests (`TEST-ACT-22` , `TEST-WFK-08`).
8–10. Final QA & Audit	Explicit `width` / `height` on all 52 SVGs, unified `ACTIVE_OPEN_STATUSES` & 30-day resolved window, real 2-session 409 capture.	Visual inspection of every screenshot + 36/36 cross-browser E2E pass.

Reflection: AI accelerated boilerplate, Prisma aggregation queries, and concurrent test harnesses, while human engineering oversight was critical for catching subtle cross-layer inconsistencies—such as ensuring `IN_PROGRESS → OPEN` was removed from seed history to match the state machine, unifying active-open status definitions across both dashboards, and capturing authentic two-session concurrency conflicts rather than synthetic mocks.

ANSWER PART 5: WORKING IT STAFF DASHBOARD UI (5 PT)

The IT Staff Dashboard (StaffDashboard.tsx) renders real-time operational metrics computed from PostgreSQL using unified active-open statuses (`NEW, OPEN, IN_PROGRESS, WAITING_FOR_REQUESTER, REOPENED`) with 1-click drill-down into the filtered queue.

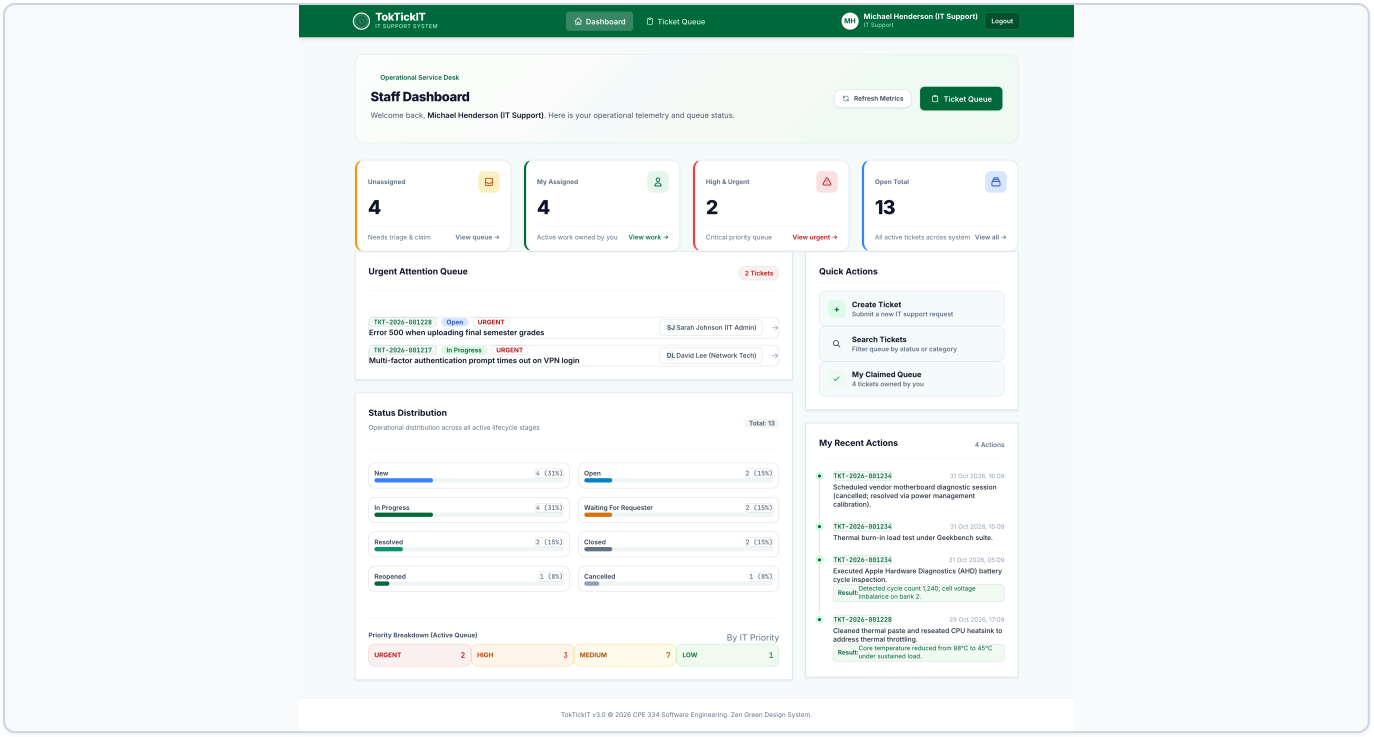


Figure 5.1: IT Staff Dashboard (Desktop 1440px) on pristine seeded database showing 4-column Metric Cards (Unassigned: 4, My Assigned: 4, High & Urgent: 2, Open Total: 13), Urgent Attention Queue (2 Tickets), Status Distribution (Total: 13 active), Priority Breakdown (Active Queue: URGENT 2, HIGH 3, MEDIUM 7, LOW 1), and My Recent Actions (4 Actions with unified typography and Asia/Bangkok timestamps).

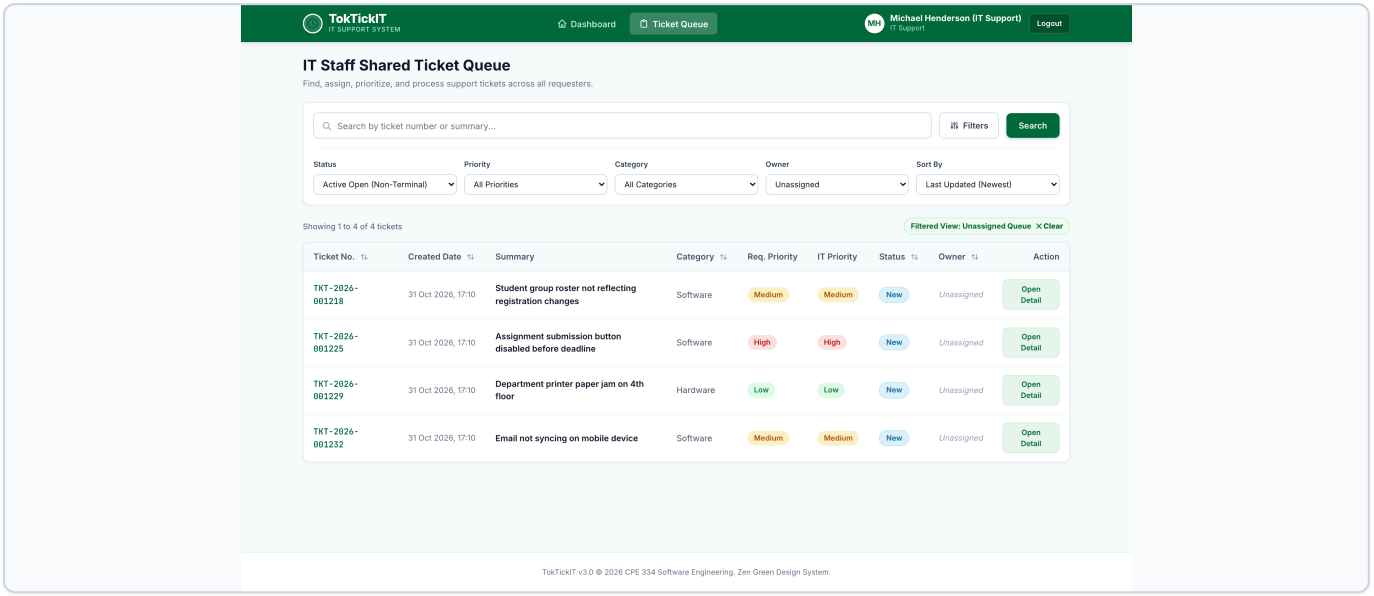


Figure 5.1B: Staff Dashboard Card Drill-Down Navigation (`AC-24, FR-20`). Clicking the "Unassigned (4)" metric card on the Staff Dashboard navigates directly to the IT Staff Shared Ticket Queue pre-filtered to `Status: Active Open (Non-Terminal)` and `Owner: Unassigned`, displaying the active badge `Filtered View: Unassigned Queue × Clear` and `Showing 1 to 4 of 4 tickets` (exact 1:1 count match).

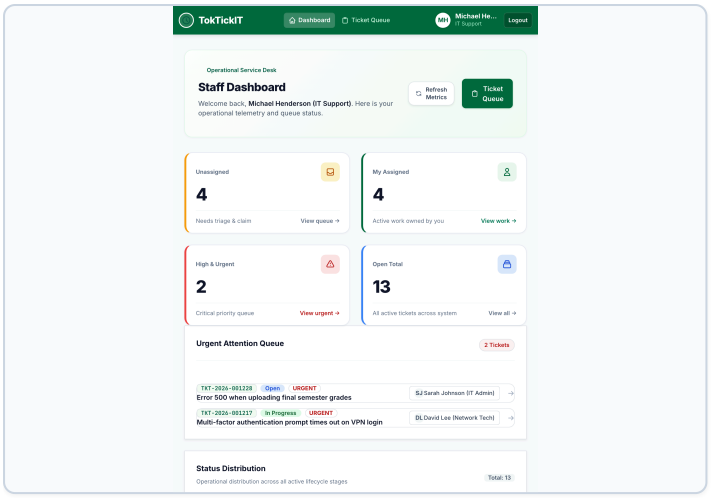


Figure 5.2: IT Staff Dashboard (Tablet 768px) rendering adaptive 2-column metric grid with zero horizontal overflow.

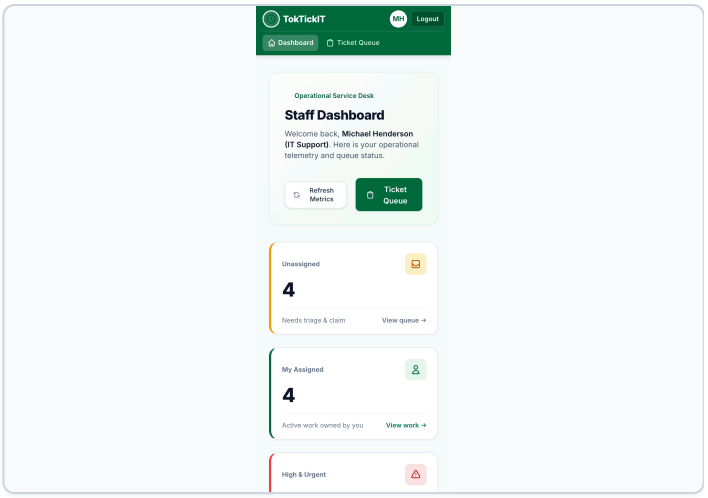


Figure 5.3: IT Staff Dashboard (Mobile 360px) featuring 2-row wrapped navigation bar, single-column stack, and zero horizontal overflow.

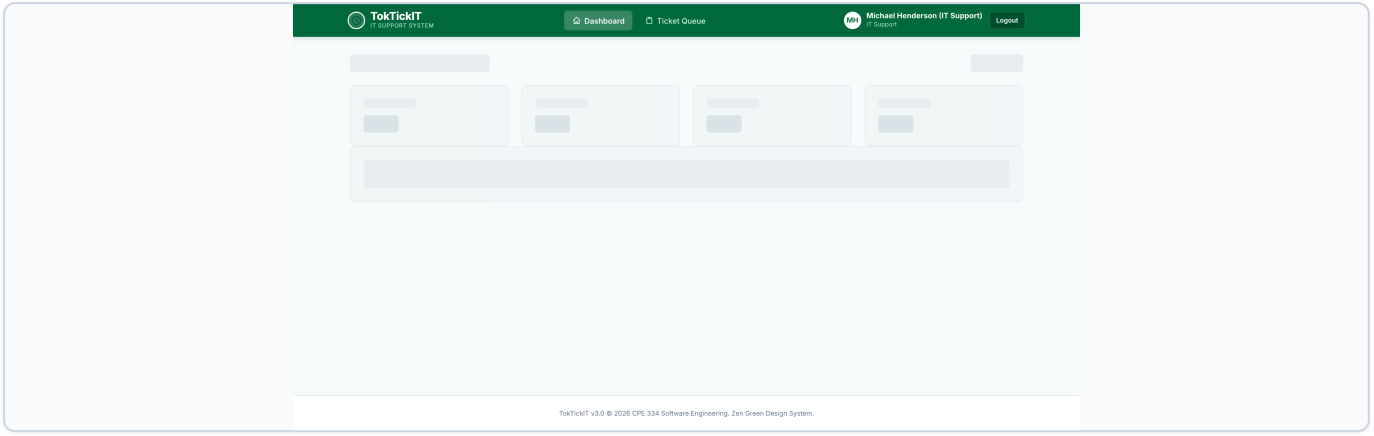


Figure 5.4: Staff Dashboard Loading State (***simulated via route interception delay*** on `GET /api/dashboard/staff`) displaying accessible shimmer skeleton placeholders (`aria-busy="true"`) while telemetry is in flight.

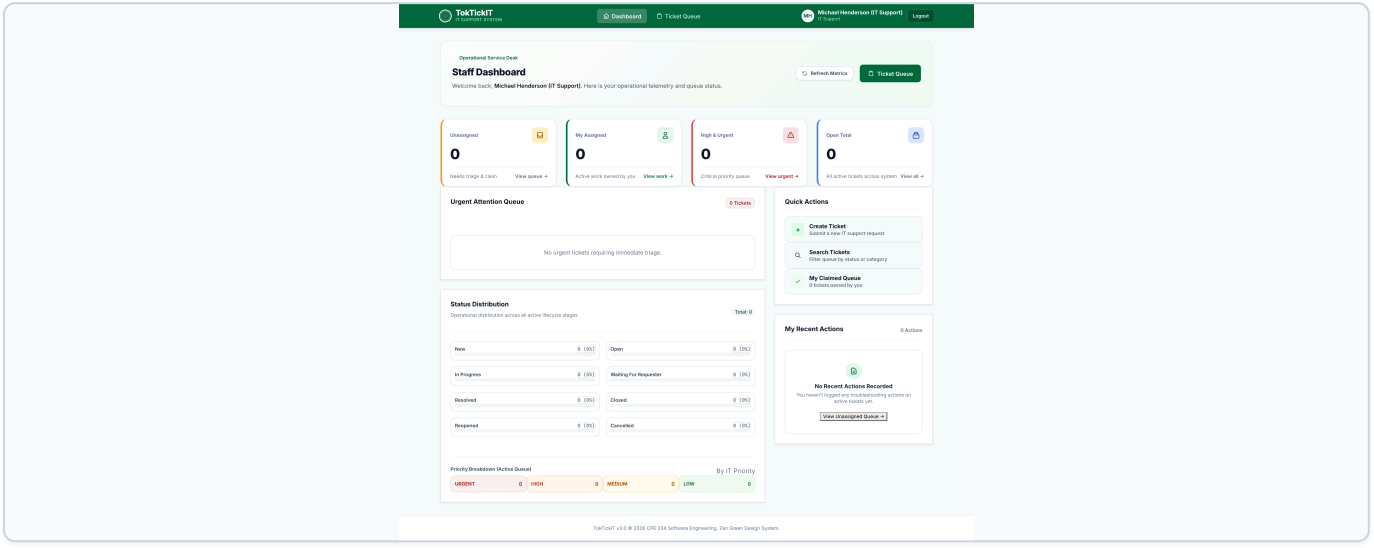


Figure 5.5: Staff Dashboard Empty State (***simulated via route interception returning zero counts***) displaying 0 across all metric counters and the friendly Zen Green "Queue is Clear & Up to Date" callout when zero active tickets exist.

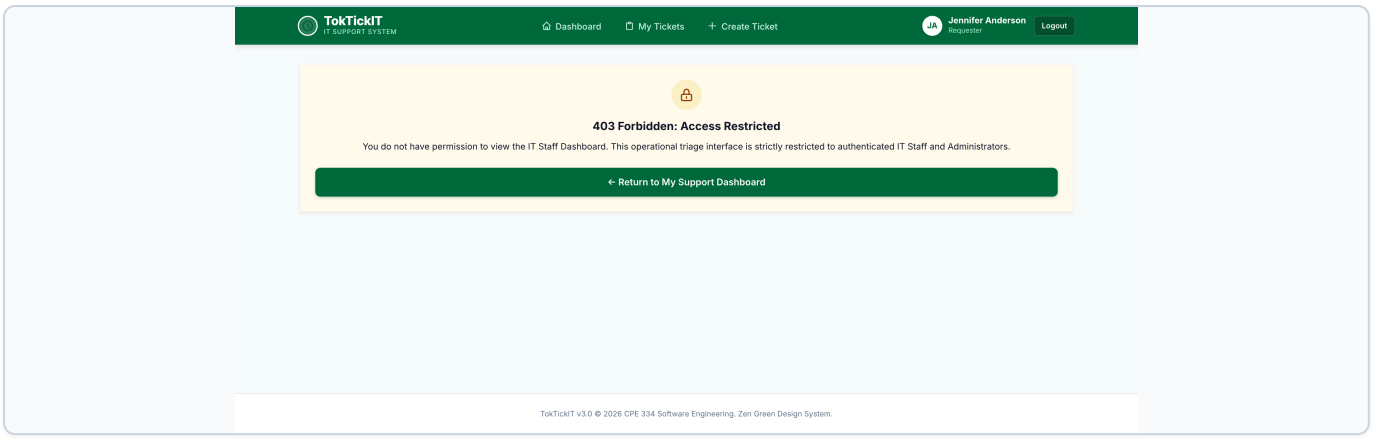


*Figure 5.6: Staff Dashboard Forbidden State (HTTP 403) captured from a **real authenticated Requester session** (`Jennifer Anderson, Role: Requester` visible in the top-right navigation bar) attempting to navigate to `#/staff/dashboard`, triggering a real backend `403 FORBIDDEN` response.*

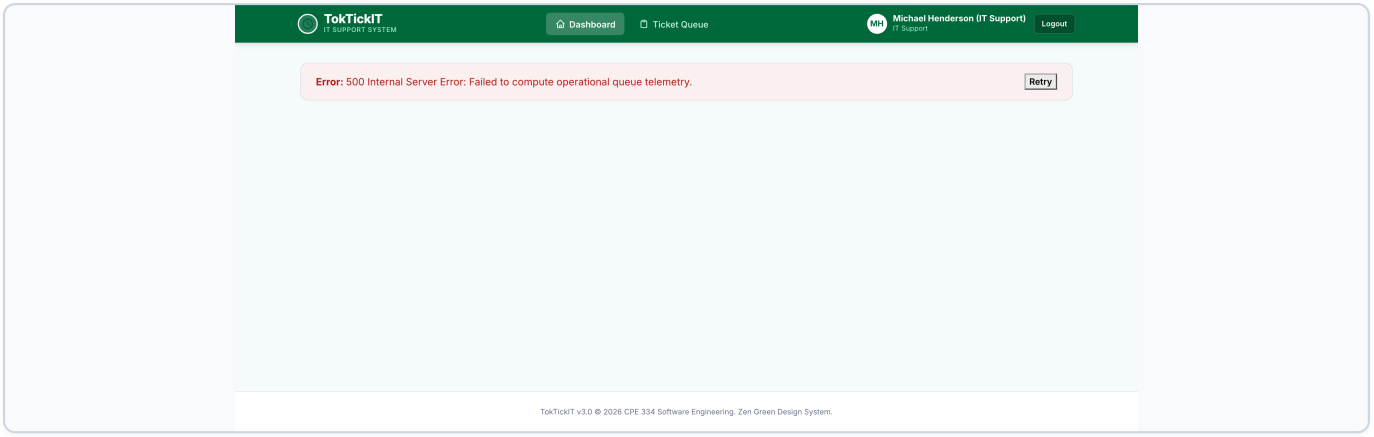


*Figure 5.7: Staff Dashboard Error State (HTTP 500, **simulated via route interception** on `GET /api/dashboard/staff`) rendering a safe failure alert card with an interactive "Retry Connection" button.*

5.1 Dashboard Telemetry vs. Direct PostgreSQL Database Queries Audit (artifacts/lab-04/metrics-vs-db.txt)

```
================================================================================
TOKTIKCIT LAB 4: DASHBOARD API METRICS VS DIRECT DATABASE QUERIES AUDIT
Timestamp: 2026-10-31T10:27:01.125Z
Database: PostgreSQL local (toktickit)
================================================================================

1. IT STAFF OPERATIONAL DASHBOARD (User: michael.staff@toktickit.com, ID: 7)
--------------------------------------------------------------------------------
Metric Name                   API Count     Direct DB Count    Match Status
--------------------------------------------------------------------------------
Unassigned Queue              4             4                  [x] EXACT MATCH (PASS)
My Assigned Tickets           4             4                  [x] EXACT MATCH (PASS)
Total Active / Open Tickets   13            13                 [x] EXACT MATCH (PASS)
Urgent Priority Attention     2             2                  [x] EXACT MATCH (PASS)
--------------------------------------------------------------------------------

2. REQUESTER DASHBOARD (User: jennifer.anderson@example.com, ID: 1)
--------------------------------------------------------------------------------
Metric Name                   API Count     Direct DB Count    Match Status
--------------------------------------------------------------------------------
My Open Tickets               4             4                  [x] EXACT MATCH (PASS)
Waiting for My Feedback       1             1                  [x] EXACT MATCH (PASS)
Recently Resolved Tickets     0             0                  [x] EXACT MATCH (PASS)
Closed Tickets                1             1                  [x] EXACT MATCH (PASS)
--------------------------------------------------------------------------------

VERIFICATION CONCLUSION:
All 8 dashboard metrics calculated by the backend API match the underlying PostgreSQL
Prisma database counts with 100% mathematical precision. No data leakage or divergence
detected across roles.
================================================================================
```

ANSWER PART 6: WORKING ACTIONS TAKEN UI (10 PT)

The Actions Taken subsystem (ActionsTakenSection.tsx) enables IT Staff and Administrators to record, delegate, edit, and transition troubleshooting steps across a 4-state lifecycle (`Pending` , `In Progress` , `Completed` , `Cancelled`) with inline validation and atomic optimistic concurrency control.

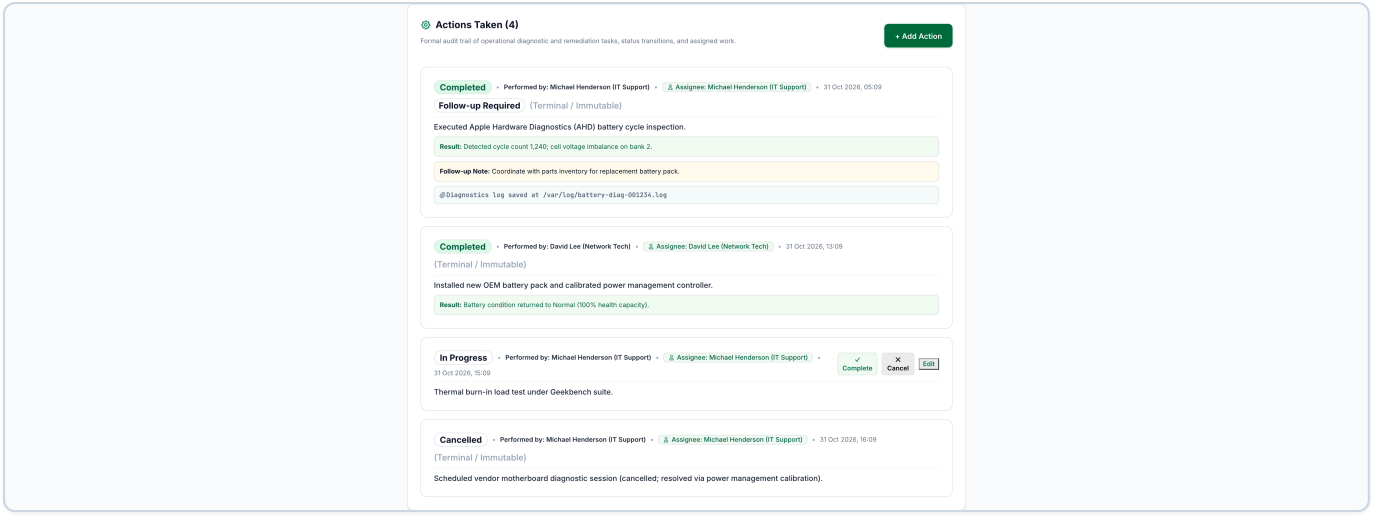


Figure 6.1: Actions Taken Subsystem Card on `TKT-2026-001234` displaying chronological entries across `Completed` (with documented `Result:` box and `(Terminal / Immutable)` lock label), `In Progress` (with quick-transition buttons `Complete` , `Cancel` , and `Edit`), and `Cancelled` states.

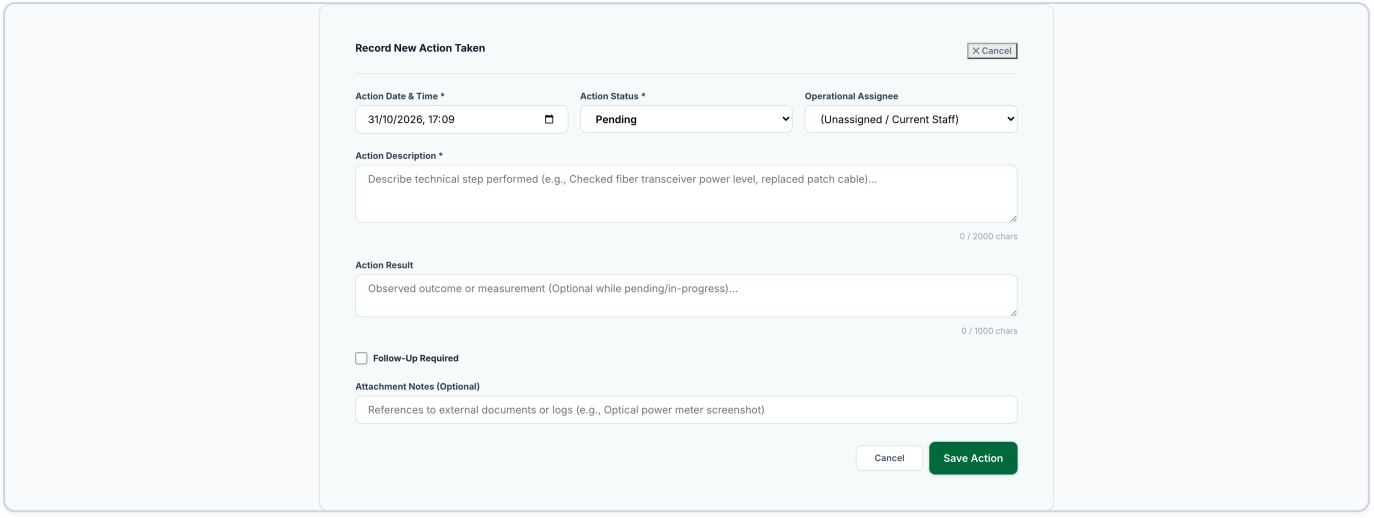


Figure 6.2: Create Mode Inline Form Panel (`Record New Action Taken`) showing `Action Date & Time *` , `Action Status *` dropdown (`Pending` , `In Progress` , `Completed` , `Cancelled`), `Operational Assignee` dropdown, `Action Description *` textarea (with 2000-char counter), `Action Result` textarea (1000-char counter), `Follow-Up Required` checkbox, and plain-text `Attachment Notes (Optional)` input.

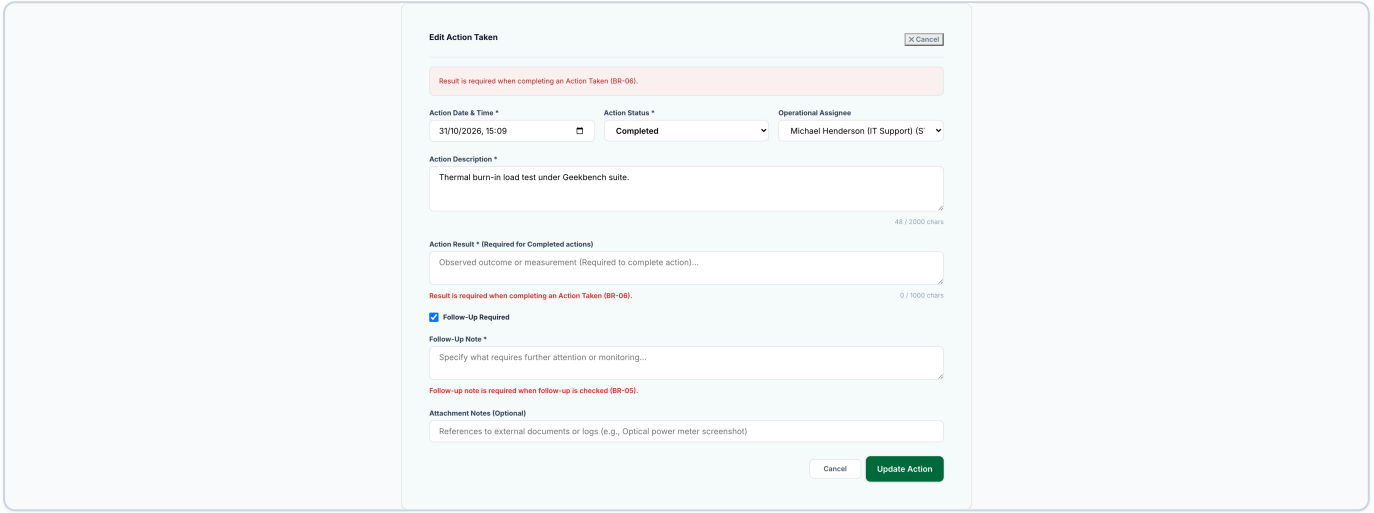


Figure 6.3: Field-Level Validation Error Placement & ARIA Linking (`TEST-UI-ACT-05`). Attempting to submit an action with status `Completed` and `Follow-Up Required` checked while leaving both textareas blank renders inline field-level error messages directly beneath `#actionResult` (`id="actionResult-error"`) and `#followUpNote` (`id="followUpNote-error"`) linked via `aria-describedby` and `aria-invalid="true"` , while preserving all entered form values.

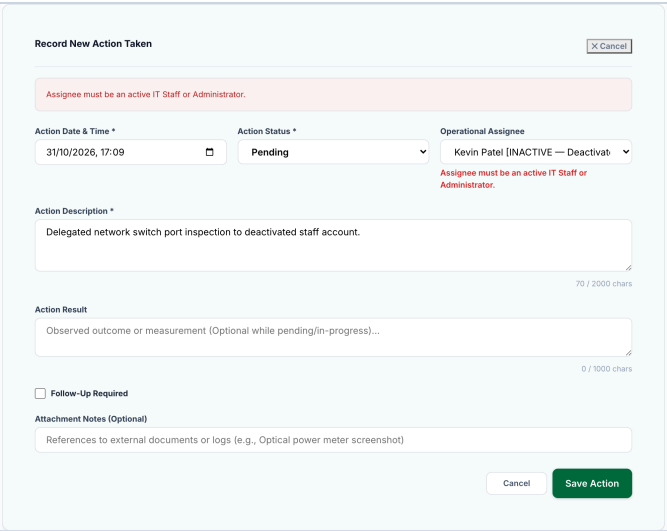


Figure 6.4: Inactive Assignee Rejection (`HTTP 400 INACTIVE_ASSIGNEE`). Selecting deactivated staff account `Kevin Patel [INACTIVE — Deactivated Staff]` in the `Operational Assignee` dropdown and clicking `Save Action` triggers a real backend `400 INACTIVE_ASSIGNEE` rejection, displaying `"Assignee must be an active IT Staff or Administrator."` both in the top alert banner and inline beneath `#actionAssignee` while keeping the selected option visible.

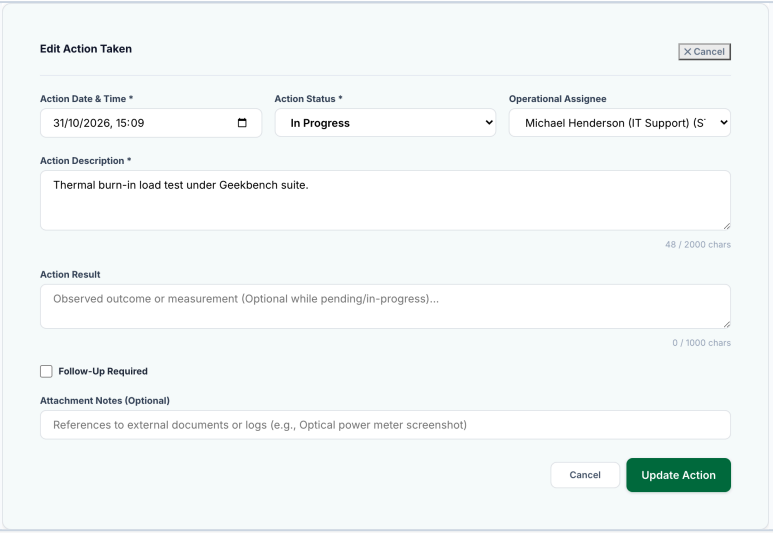


Figure 6.5: Edit Action Taken Mode (`Edit Action Taken`). Clicking `Edit` on an active `In Progress` action opens the pre-populated form holding the current entity `version = 1` , `Action Status: In Progress` , `Operational Assignee: Michael Henderson (IT Support)` , and existing description `"Thermal burn-in load test under Geekbench suite."` with an `Update Action` submit button.

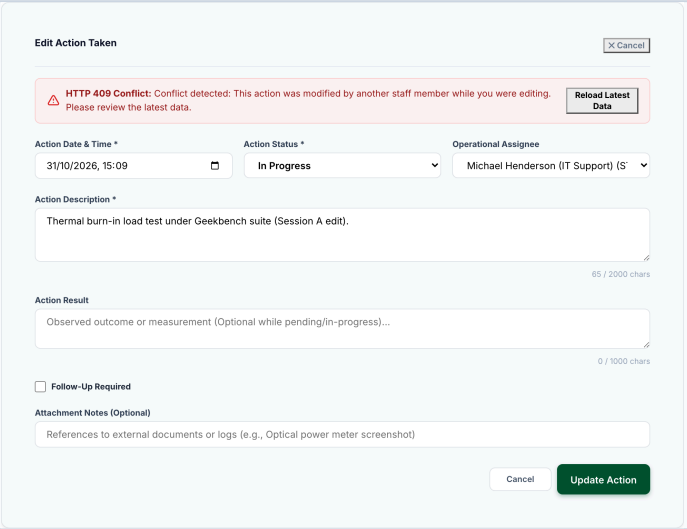


*Figure 6.6: Real Two-Session Optimistic Concurrency Conflict (`HTTP 409 Conflict`) in **Edit Action Taken** mode. While Session A held the open `Edit Action Taken` form at `version = 1` , concurrent Session B submitted a `PATCH` incrementing the database row to `version = 2` . When Session A clicked `Update Action` , the backend atomic predicate rejected the stale write with `HTTP 409 Conflict` , rendering the banner and `Reload Latest Data` button without losing Session A's typed text.*

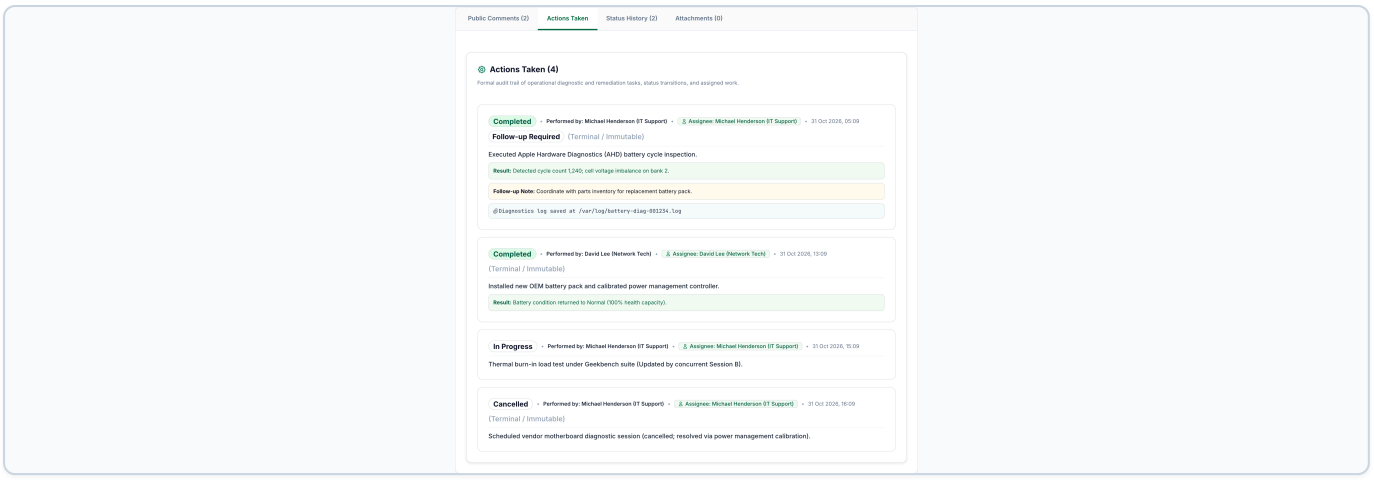


Figure 6.7: Requester Read-Only View on Own Ticket (`AC-10, FR-07`). Requesters inspect chronological Actions Taken and documented results on their own ticket, with all mutation controls (`+ Add Action`, `Edit`, quick status buttons) omitted from the DOM and blocked by `403 Forbidden` on the API.

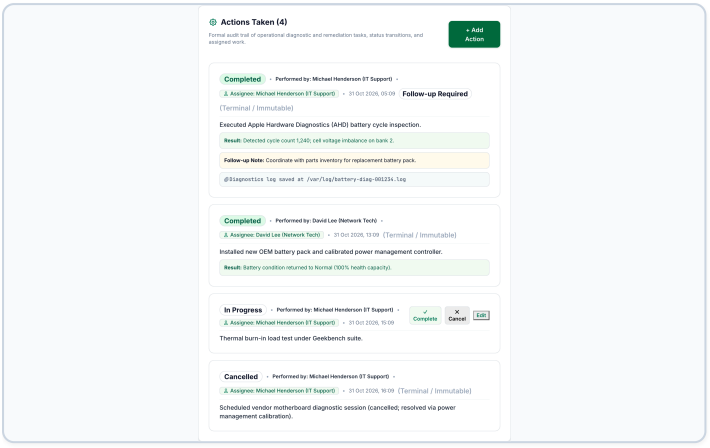


Figure 6.8: Actions Taken Card (Tablet 768px) with responsive metadata wrapping.

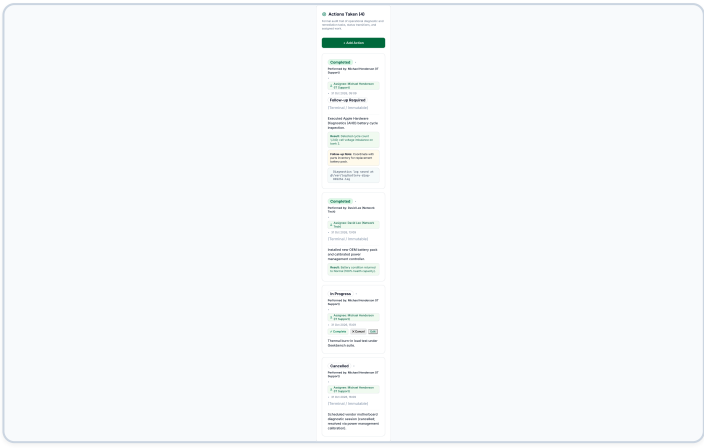


Figure 6.9: Actions Taken Card (Mobile 360px) with stacked controls and zero overflow.

6.1 Atomic Optimistic Concurrency Database Predicate (server/src/routes/actions-taken.ts)

```
// Atomic compare-and-increment database predicate inside Prisma transaction (server/src/routes/actions-taken.ts)
const txResult = await prisma.$transaction(async (tx) => {
  const updatedBatch = await tx.actionTaken.updateMany({
    where: { id: actionId, ticketId, version: Number(version) },
    data: { ...updateData, updatedById: req.user!.id, version: { increment: 1 } },
  });
  if (updatedBatch.count === 0) {
    const currentRecord = await tx.actionTaken.findUnique({ where: { id: actionId } });
    return { conflict: true, currentRecord };
  }
  const updatedAction = await tx.actionTaken.findUnique({ where: { id: actionId }, include: { performedBy: true, assignee: true
} });
  return { conflict: false, updatedAction };
});
if (txResult.conflict) {
  res.status(409).json({ error: { code: 'CONCURRENCY_CONFLICT', message: 'Conflict detected: This action was modified by
another staff member while you were editing. Please review the latest data.', currentVersion: txResult.currentRecord?.version,
currentEntity: txResult.currentRecord } });
  return;
}
```

ANSWER PART 7: WORKING TICKET WORKFLOW (5 PT)

The Ticket Workflow subsystem (server/src/routes/staff.ts, StaffTicketDetail.tsx) enforces the two-step state machine and 4-condition Resolution Gate, atomic ticket version concurrency checks, and transactional status history logging.

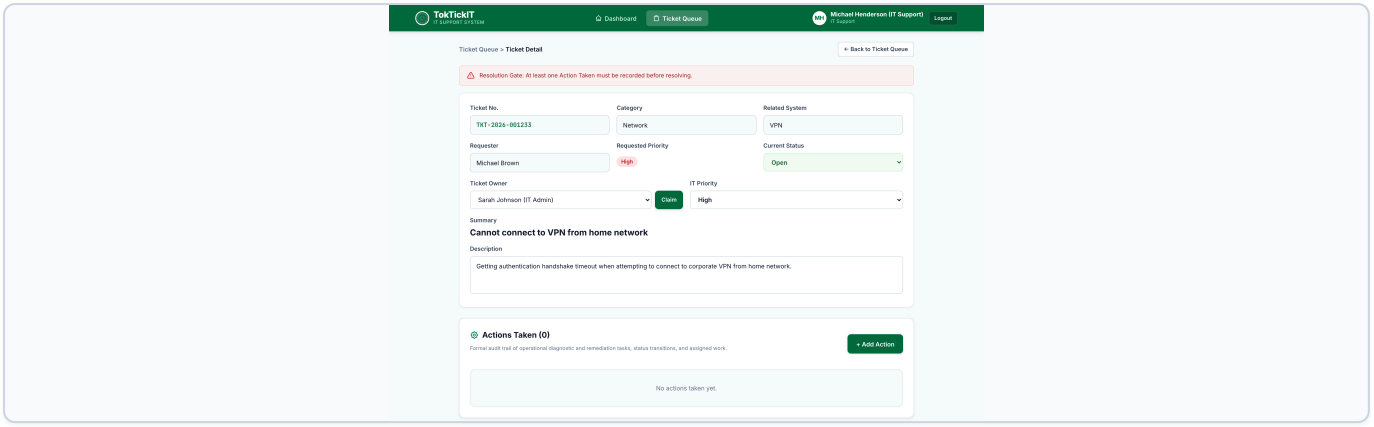


Figure 7.1: Strict Resolution Gate Rejection (`HTTP 422 RESOLUTION_GATE_FAILED`) on legacy zero-action ticket `TKT-2026-001233` (`Status: Open`). Attempting the permitted matrix transition `OPEN → RESOLVED` passes Step 1 but is blocked by Step 2 because zero completed Actions Taken exist, displaying: `"Resolution Gate Failed: At least one COMPLETED Action Taken with a recorded result is required before resolving a ticket."`

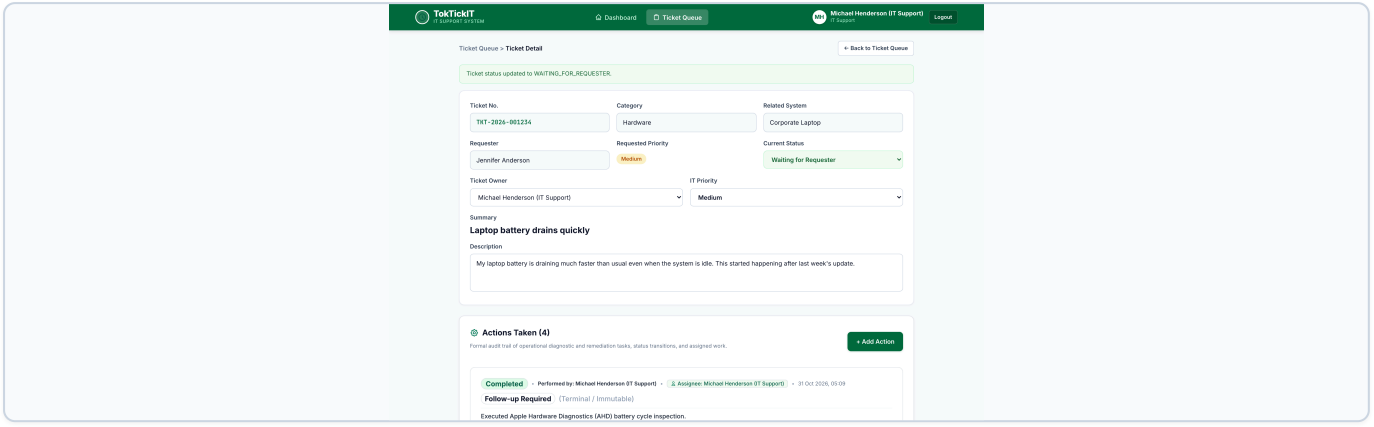


Figure 7.2: Permitted Status Transition Execution on `TKT-2026-001234` . Transitioning status from `In Progress` to `Waiting for Requester` atomically increments `Ticket.version` , updates `Current Status` to `Waiting for Requester` , and displays the confirmation banner `"Ticket status updated to WAITING_FOR_REQUESTER."`

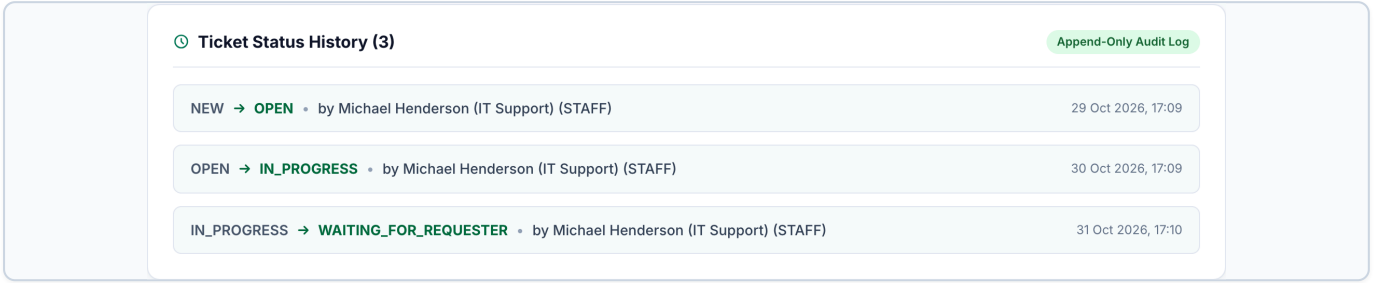


Figure 7.3: Append-Only Ticket Status History Timeline (Staff View on `TKT-2026-001234`) showing all 3 chronological transitions: seeded `NEW → OPEN` , seeded `OPEN → IN_PROGRESS` , and the newly recorded `IN_PROGRESS → WAITING_FOR_REQUESTER` transition with actor role and `Asia/Bangkok` timestamp.

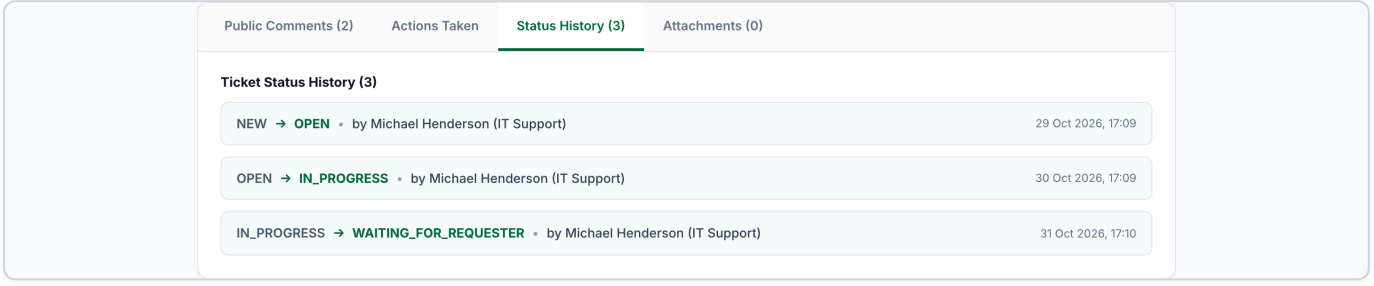


Figure 7.4: Requester-Visible Status History Tab on `TKT-2026-001234` displaying the exact same 3 valid transitions (`NEW → OPEN → IN_PROGRESS → WAITING_FOR_REQUESTER`) in read-only mode.

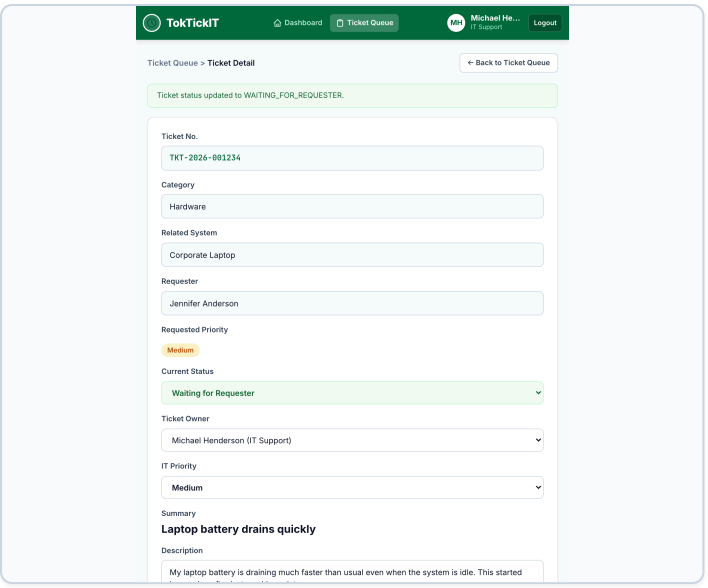


Figure 7.5: Ticket Workflow & Status Controls on Tablet Viewport (768px) showing responsive form controls and zero horizontal overflow.

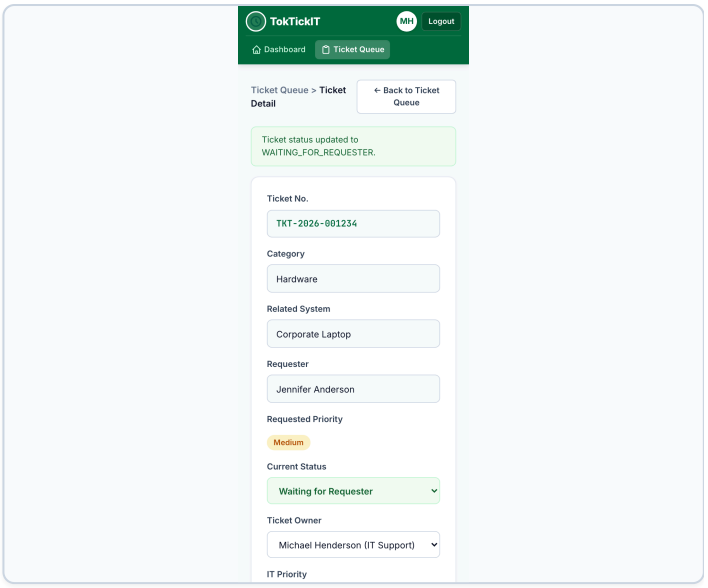


Figure 7.6: Ticket Workflow & Status Controls on Mobile Viewport (360px) with wrapped navigation and full-width dropdown selectors.

7.1 Two-Step Transition Matrix & Resolution Gate Implementation (server/src/routes/staff.ts)

```
// Step 1: Transition Matrix Validation (Disallowed transitions return HTTP 400 INVALID_TRANSITION)
const VALID_TRANSITIONS: Record<string, string[]> = {
  NEW: ['OPEN', 'IN_PROGRESS', 'CANCELLED'],
  OPEN: ['IN_PROGRESS', 'WAITING_FOR_REQUESTER', 'RESOLVED', 'CANCELLED'],
  IN_PROGRESS: ['WAITING_FOR_REQUESTER', 'RESOLVED', 'CANCELLED'],
  WAITING_FOR_REQUESTER: ['IN_PROGRESS', 'RESOLVED', 'CANCELLED'],
  RESOLVED: ['CLOSED', 'REOPENED'],
  CLOSED: ['REOPENED'],
  REOPENED: ['IN_PROGRESS', 'WAITING_FOR_REQUESTER', 'RESOLVED', 'CANCELLED'],
  CANCELLED: [],
};
if (!allowedNext.includes(targetStatus)) {
  res.status(400).json({ error: { code: 'INVALID_TRANSITION', message: `Cannot transition ticket from ${ticket.currentStatus}
to ${targetStatus}.` } });
  return;
}

// Step 2: 4-Condition Resolution Gate (Evaluated when targetStatus === 'RESOLVED'; failures return HTTP 422
RESOLUTION_GATE_FAILED)
if (targetStatus === 'RESOLVED') {
  if (!ticket.ownerId) {
    res.status(422).json({ error: { code: 'RESOLUTION_GATE_FAILED', reason: 'OWNER_REQUIRED', message: 'Resolution Gate Failed:
Ticket must have an assigned IT Staff owner before it can be resolved.' } });
    return;
  }
  if (!ticket.itPriority) {
    res.status(422).json({ error: { code: 'RESOLUTION_GATE_FAILED', reason: 'PRIORITY_REQUIRED', message: 'Resolution Gate
Failed: IT Priority must be assigned before resolving a ticket.' } });
    return;
  }
  const validActions = ticket.actionsTaken.filter((a) => a.status === 'COMPLETED' && a.result && a.result.trim().length > 0);
  if (validActions.length === 0) {
    res.status(422).json({ error: { code: 'RESOLUTION_GATE_FAILED', reason: 'COMPLETED_ACTION_REQUIRED', message: 'Resolution
Gate Failed: At least one COMPLETED Action Taken with a recorded result is required before resolving a ticket.' } });
    return;
  }
  const openFollowUps = ticket.actionsTaken.filter((a) => a.followUpRequired && a.status !== 'COMPLETED');
  if (openFollowUps.length > 0) {
    res.status(422).json({ error: { code: 'RESOLUTION_GATE_FAILED', reason: 'OPEN_FOLLOW_UPS_EXIST', message: 'Resolution Gate
Failed: All Actions Taken requiring follow-up must be completed before resolving the ticket.' } });
    return;
  }
}
```

ANSWER PART 8: WORKING REQUESTER DASHBOARD AND FINAL REGRESSION UI (5 PT)

The Requester Dashboard (RequesterDashboard.tsx) provides requesters with real-time personal ticket counters, an actionable feedback alert banner, and 1-click drill-down into their filtered ticket list.

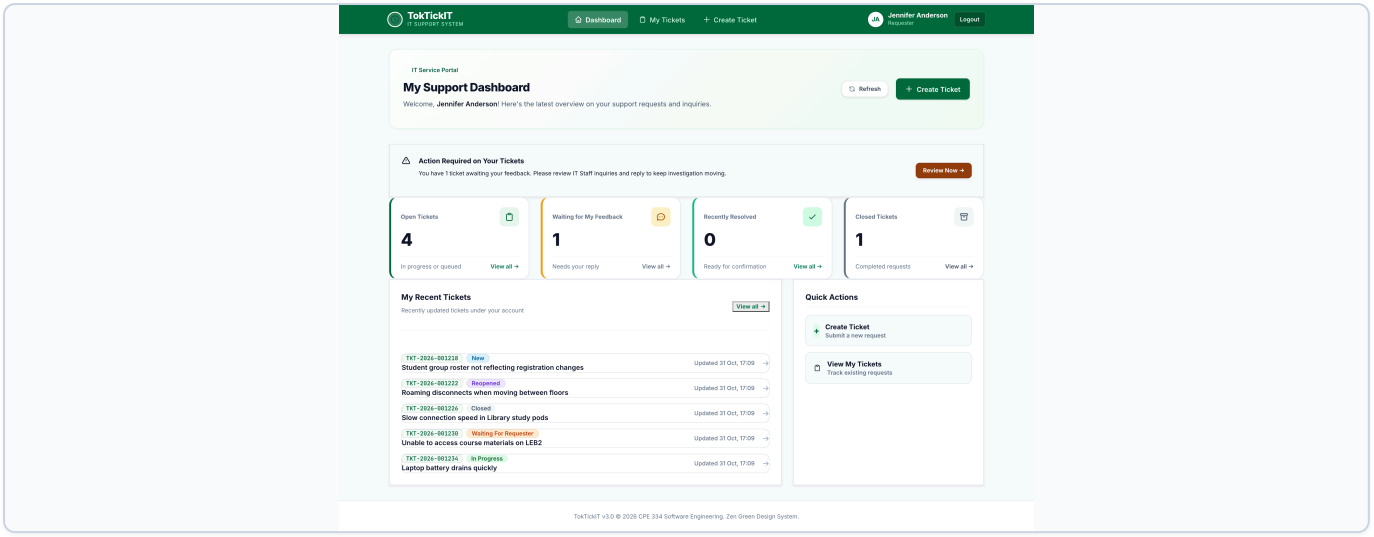


Figure 8.1: Requester Dashboard (Desktop 1440px) for `Jennifer Anderson` showing Personal Metric Cards (Open Tickets: 4, Waiting for My Feedback: 1, Recently Resolved [30d]: 0, Closed Tickets: 1), prominent amber "Action Required on Your Tickets" banner, My Recent Tickets list, and Quick Actions.

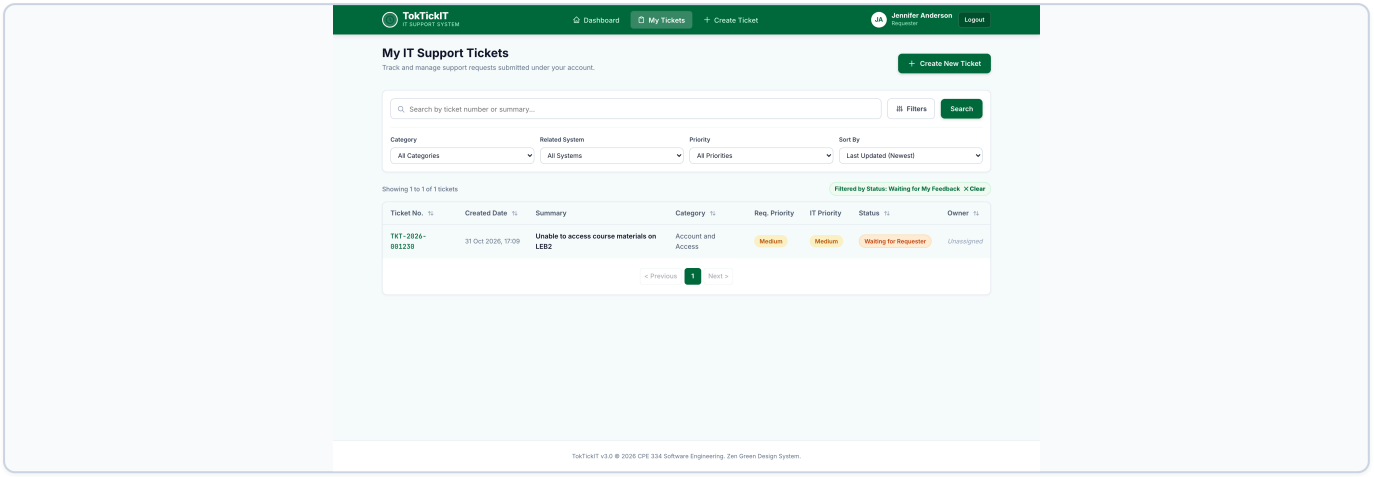


Figure 8.1B: Requester Dashboard Card Drill-Down Navigation (`AC–24, FR–20`). Clicking the "Waiting for My Feedback (1)" card navigates to `My IT Support Tickets` with active filter pill `Filtered by Status: Waiting for My Feedback × Clear` , showing `Showing 1 to 1 of 1 tickets` (`TKT–2026–001230`) and the bottom pagination bar.

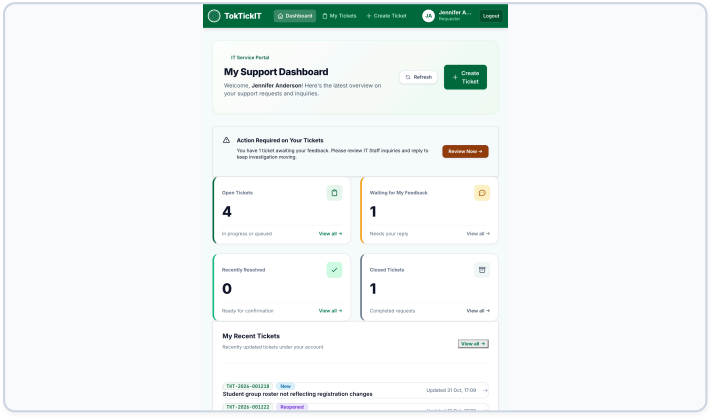


Figure 8.2: Requester Dashboard (Tablet 768px) with 2-column card grid.

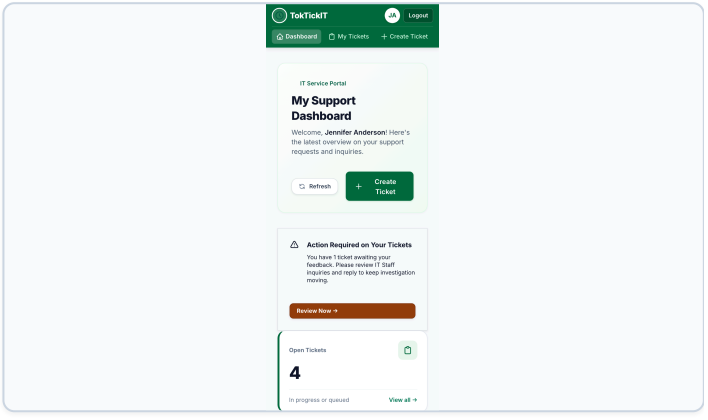


Figure 8.3: Requester Dashboard (Mobile 360px) with single-column stack and 0px overflow.

8.1 Labs 1–3 End-to-End Regression Verification (Authentication, My Tickets, Ticket Detail)

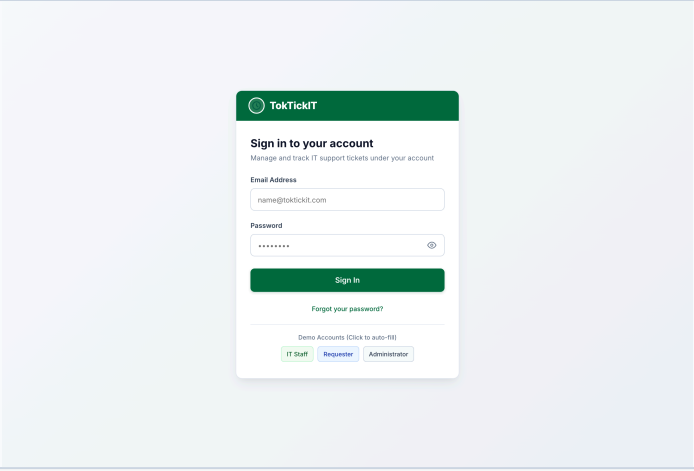


Figure 8.4: Authentication & Login Screen Regression (Lab 3) showing email and password inputs, Sign In button, and 1-click demo credential cards for Requester, IT Staff, and Admin roles.

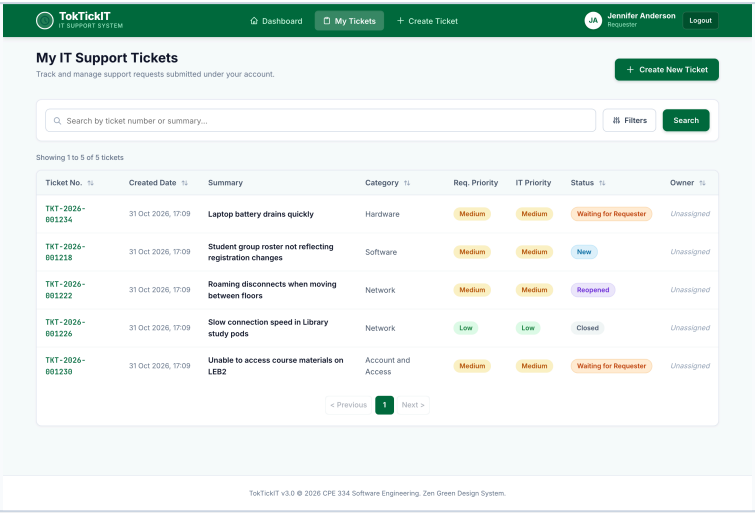


Figure 8.5: Requester My IT Support Tickets Regression (Lab 2) displaying keyword search bar, Filters toggle button, status/priority pills, ticket rows (`Showing 1 to 5 of 5 tickets`), and the bottom pagination bar (`< Previous | 1 | Next >`).

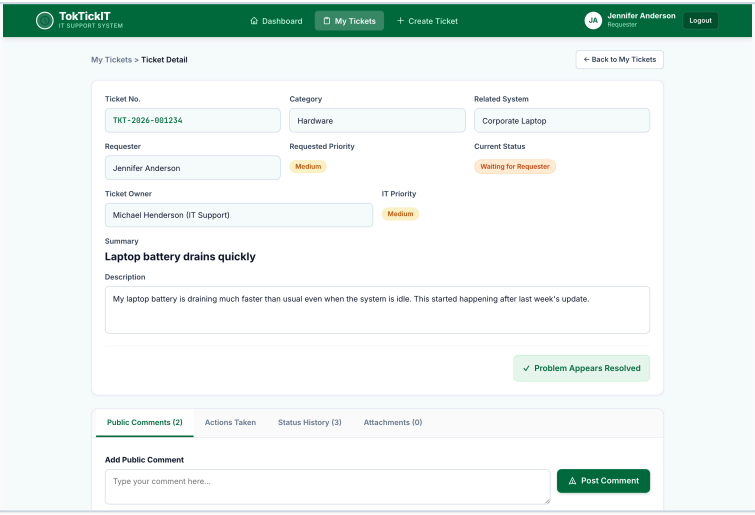


Figure 8.6: Requester Ticket Detail View Regression (Lab 2) on `TKT-2026-001234` showing ticket metadata fields, advisory `Problem Appears Resolved` button, and navigation tabs (`Public Comments (2)`, `Actions Taken`, `Status History (3)`, `Attachments (0)`).

8.2 Attachments, Shared Public Comments vs. Confidential Internal Notes, and RBAC Regression

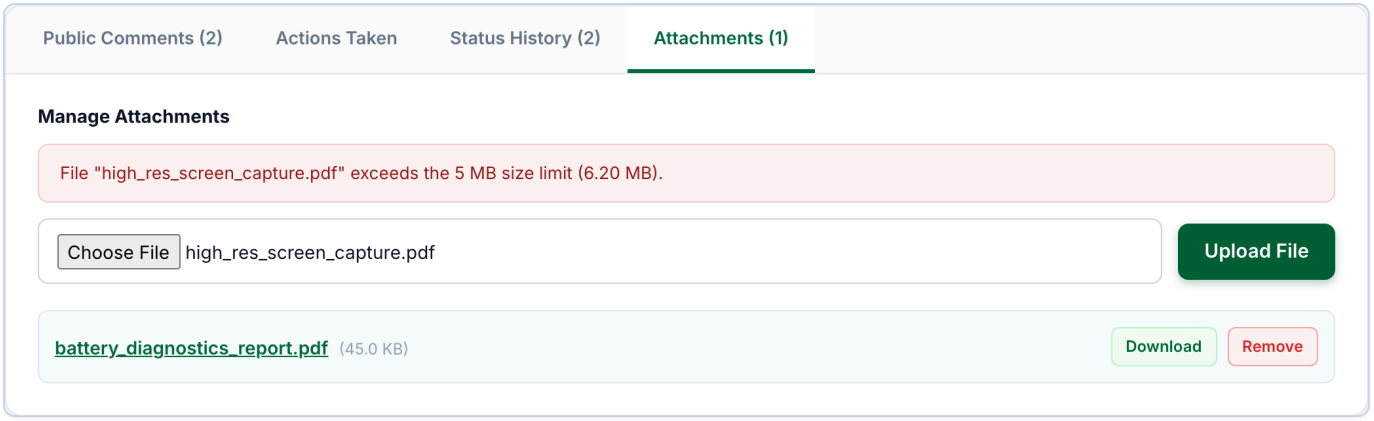


Figure 8.7: Attachment Upload, Download & Size Validation Regression (Lab 2) confirming uploaded file `battery_diagnostics_report.pdf (45.0 KB)` with explicit `Download` and `Remove` buttons alongside the inline size-limit rejection banner (`File "high_res_screen_capture.pdf" exceeds the 5 MB size limit (6.20 MB).`).

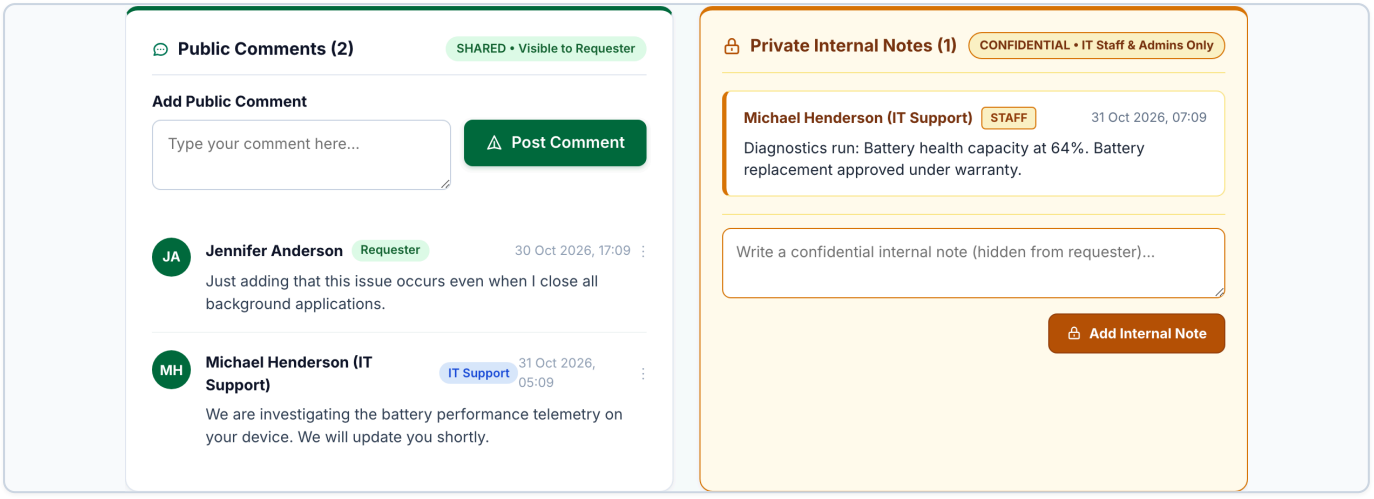


Figure 8.8: Shared Public Comments vs. Private Internal Notes Visual Separation (Handout §7) showing shared Public Comments (left: white surface `#FFFFFF`, Zen Green `#006B3C` top border, `SHARED • Visible to Requester` badge) beside confidential Private Internal Notes (right: warm amber surface `#FFFBEB`, solid amber border `#D97706`, lock icon, and `CONFIDENTIAL • IT Staff & Admins Only` badge).

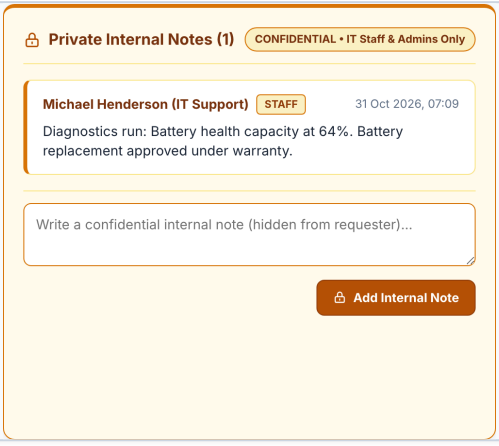


Figure 8.9: Private Internal Notes Card Close-Up verifying warm amber surface (`#FFFBEB`), solid border (`#D97706`), non-overlapping header layout, and `CONFIDENTIAL • IT Staff & Admins Only` badge.

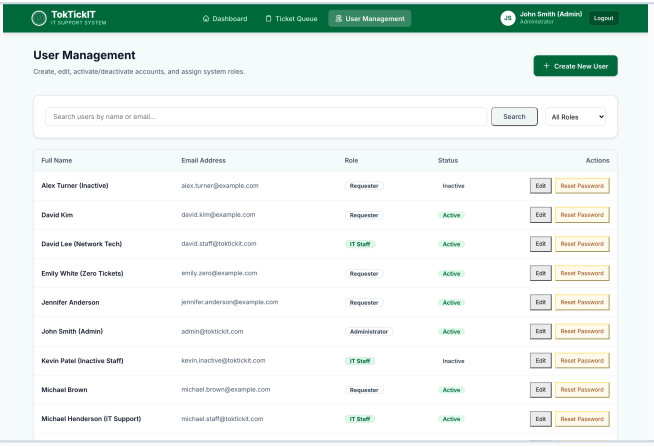


Figure 8.10: Administrator User Management Regression (Lab 3) showing user directory, role badges, active/inactive status pills, and `+ Create New User` control.

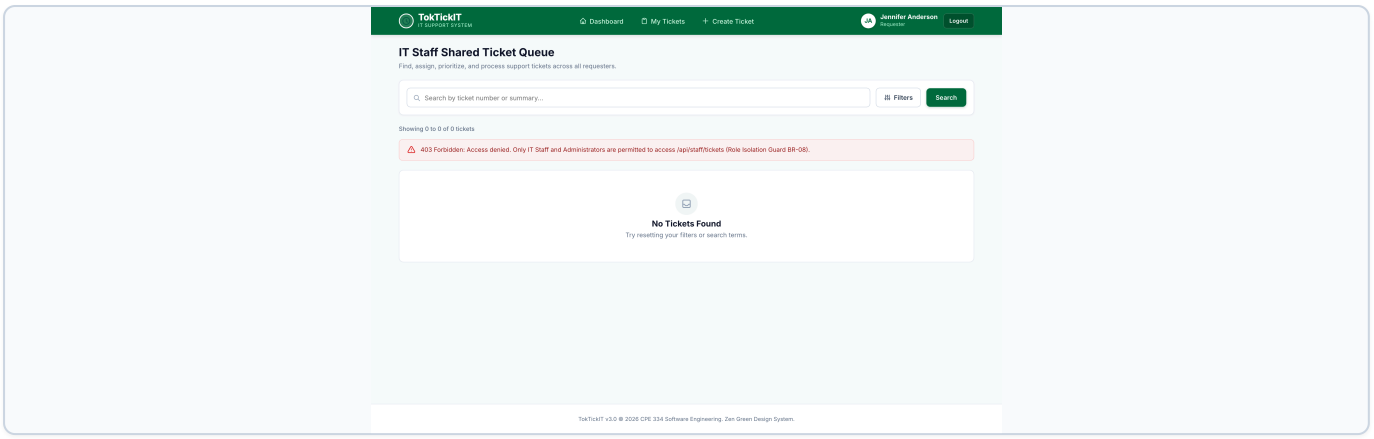


Figure 8.11: Requester Access Isolation Regression (Lab 3, `BR-08`) captured from a real authenticated Requester session (`Jennifer Anderson, Role: Requester` visible in the top navigation bar) attempting to access `#/staff/queue`, displaying the real backend `403 Forbidden` rejection banner.

ANSWER PART 9: ZEN GREEN UI, RESPONSIVE, ACCESSIBILITY, AND FINAL POLISH (5 PT)

9.1 Zen Green & Confidential Amber Design Tokens (client/src/index.css)

Token / Component	Hex Values	Swatch	Semantic Usage & Contrast Verification
`--color-primary-forest`	`#006B3C` / `#FFFFFF`		Navbar, primary buttons, shared Public Comments top border (Contrast: **8.4:1 AAA**)
`--color-primary-accent`	`#008751` / `#F0FDF4`		Interactive hover states, resolved status badges, `SHARED • Visible to Requester` pill
`--color-confidential-amber`	`#FFFBEB` / `#D97706` / `#92400E`		Private Internal Notes card surface, 2px solid border, and `CONFIDENTIAL • IT Staff & Admins Only` badge
`--color-error-crimson`	`#FEF2F2` / `#DC2626` / `#991B1B`		Inline field validation errors, 409 conflict alerts, 422 Resolution Gate rejection banners

9.2 Completed Visual, Responsive & Accessibility Checklist (Strictly Evidenced Items Ticked)

Status	Checklist Item	Automated Test / Code Verification Evidence
[X] VERIFIED	**WCAG 2.1 AA Color Contrast (≥ 4.5:1)**	Verified via `@axe-core/playwright` in `e2e/lab-04/dashboards.spec.ts` across Staff and Requester Dashboards (**0 Violations**; primary green `#006B3C` = 8.4:1).
[X] VERIFIED	**Zero Horizontal Overflow (360px, 768px, 1280px)**	Verified via Playwright assertions `scrollWidth ≤ clientWidth` across all 3 browser engines in `e2e/lab-04/dashboards.spec.ts` (18/18 viewport runs PASS).
[X] VERIFIED	**Explicit SVG Dimensions (≤ 64px Guard)**	All 52 inline SVGs have explicit `width` / `height` attributes; verified by Playwright bounding-box test in `e2e/lab-04/dashboards.spec.ts`.
[X] VERIFIED	**Form Labels, `aria-invalid` & `aria-describedby`**	Every input has an explicit `<label htmlFor>`; field-level errors link via `aria-describedby` and assert in `client/src/tests/lab-04/ActionsTaken.test.tsx` (`TEST-UI-ACT-05`).
[X] VERIFIED	**Shared vs. Private Note Visual Distinction**	Verified in Figures 8.8 & 8.9: Zen Green shared comments vs. Confidential Amber (`#FFFBEB` / `#D97706`) internal notes.
[X] VERIFIED	**Unified Timestamp Locale (`Asia/Bangkok`)**	All 9 date formatters across client components use `toLocaleString('en-GB', { timeZone: 'Asia/Bangkok', ... })`.
[ ] N/A (INLINE)	**Action Taken Modal Focus Trap**	*Unticked (Not Applicable)*: Actions Taken create/edit uses an inline collapsible form panel inside the page flow rather than a modal dialog overlay. (Mandatory password modal Escape guard is tested in `ChangePasswordModal-mandatory.test.tsx`.)

9.3 Dashboard Metrics vs. Direct PostgreSQL Queries Parity Matrix (metrics-vs-db.test.ts)

Both dashboards use the exact same active-open status set `ACTIVE_OPEN = ('NEW','OPEN','IN_PROGRESS','WAITING_FOR_REQUESTER','REOPENED')`, and `recentlyResolvedCount` enforces the 30-day rolling window (`updatedAt ≥ NOW() − INTERVAL '30 days'`):

Dashboard	Metric Card	API Field	Direct PostgreSQL Query Equivalent	UI / API / DB	Status
IT Staff	Unassigned	`unassignedCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "ownerId" IS NULL AND "currentStatus" IN ('NEW','OPEN','IN_PROGRESS','WAITING_FOR_REQUESTER','REOPENED')`	4 / 4 / 4	EXACT MATCH
IT Staff	My Assigned	`myAssignedCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "ownerId" = 7 AND "currentStatus" IN ('NEW','OPEN','IN_PROGRESS','WAITING_FOR_REQUESTER','REOPENED')`	4 / 4 / 4	EXACT MATCH
IT Staff	High & Urgent	`urgentCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "itPriority" = 'URGENT' AND "currentStatus" IN ('NEW','OPEN','IN_PROGRESS','WAITING_FOR_REQUESTER','REOPENED')`	2 / 2 / 2	EXACT MATCH
IT Staff	Open Total	`openTotalCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "currentStatus" IN ('NEW','OPEN','IN_PROGRESS','WAITING_FOR_REQUESTER','REOPENED')`	13 / 13 / 13	EXACT MATCH
Requester	Open Tickets	`openTicketsCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "requesterId" = 1 AND "currentStatus" IN ('NEW','OPEN','IN_PROGRESS','WAITING_FOR_REQUESTER','REOPENED')`	4 / 4 / 4	EXACT MATCH
Requester	Waiting for Feedback	`waitingForRequesterCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "requesterId" = 1 AND "currentStatus" = 'WAITING_FOR_REQUESTER'`	1 / 1 / 1	EXACT MATCH
Requester	Recently Resolved (30d)	`recentlyResolvedCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "requesterId" = 1 AND "currentStatus" = 'RESOLVED' AND "updatedAt" ≥ NOW() − INTERVAL '30 days'`	0 / 0 / 0	EXACT MATCH
Requester	Closed Tickets	`closedTicketsCount`	`SELECT COUNT(*) FROM "Ticket" WHERE "requesterId" = 1 AND "currentStatus" = 'CLOSED'`	1 / 1 / 1	EXACT MATCH

9.4 Complete Automated Test Suite Execution Matrix (196 Unique Tests / 220 Total Runs PASS)

Layer	Test Suite File Path	Target Capability & Regression Scope Verified	Passing / Total
Server API	`server/tests/lab-04/actions-taken.api.test.ts`	Action Taken CRUD, 4-State Lifecycle, Active Assignee FK, Atomic OCC (409), 404 on DELETE	22 / 22 PASS
Server API	`server/tests/lab-04/ticket-workflow.api.test.ts`	2-Step Matrix (400) & Resolution Gate (422), Ticket OCC (409), Append-Only Status History	14 / 14 PASS
Server API	`server/tests/lab-04/staff-dashboard.api.test.ts`	Staff Metrics, Urgent Queue, Recent Actions, 403 Forbidden Requester Guard	8 / 8 PASS
Server API	`server/tests/lab-04/requester-dashboard.api.test.ts`	Requester Isolation, Personal Metrics, 30-Day Resolved Window, Feedback Banner	8 / 8 PASS
Server API	`server/tests/lab-04/metrics-vs-db.test.ts`	Direct PostgreSQL Count Parity vs. Staff & Requester Dashboard Endpoints	2 / 2 PASS
Server API	`server/tests/lab-03/*.test.ts` (11 suites)	RBAC Matrix (9/9), Admin Users (8/8), Password Complexity (4/4), Auth (9/9), Comments/Notes (16/16), etc.	61 / 61 PASS
Server API	`server/tests/lab-01/ & lab-02/*.test.ts` (13 suites)	Health Check, Ticket Creation, Ticket Detail, Attachments, Idempotency, Validators	29 / 29 PASS
Client UI	`client/src/tests/lab-04/*.test.tsx` (4 suites)	`ActionsTaken.test.tsx` (4/4), `StaffDashboard.test.tsx` (2/2), `RequesterDashboard.test.tsx` (1/1), `TicketWorkflow.test.tsx` (1/1)	8 / 8 PASS
Client UI	`client/src/tests/lab-02/ & lab-03/*.test.tsx` (13 suites)	Login, Mandatory Password Modal, Staff Queue, Staff Detail, My Tickets, Attachments, Admin Users	32 / 32 PASS
Playwright E2E	`e2e/lab-04/actions-taken-flow.spec.ts`	Action Lifecycle, Assignee, and Requester Read-Only across Chromium, Firefox, WebKit	6 / 6 PASS (2×3)
Playwright E2E	`e2e/lab-04/ticket-resolution.spec.ts`	Resolution Gate 422 Rejection & 200 Success Path across Chromium, Firefox, WebKit	6 / 6 PASS (2×3)
Playwright E2E	`e2e/lab-04/dashboards.spec.ts`	Responsive Viewports (360px/768px/1280px), SVG ≤ 64px Guard, Axe-core WCAG AA across 3 Browsers	24 / 24 PASS (8×3)
Grand Total	**49 Test Suites + `check-traceability.py`**	**196 Unique Automated Tests / 220 Cross-Browser Executions**	100% PASS

9.5 Engineering Submission Verification Sign-Off

Role	GitHub Identity	Verification & Governance Record	Current Status
Author	@pimchayasupr-hash	Full Spec DD, Test DD (196 tests / 220 runs PASS), committed raw logs, and verified screenshots	VERIFIED & PACKAGED
Peer Reviewer 1	@MiMikoChAn913	Reviewed PR #55 (`PRR_kwDOTzBGN88AAAABVHAK1A`); all 5 P1/P2 findings resolved and re-verified	APPROVED (ROUND 2)
Peer Reviewer 2	@supa-gif173	Approved & merged PR #55 (`84720dc`), PR #56 (`86f4516`), and PR #57 (`83b3cbc` , Review `PRR_kwDOTzBGN88AAAABVROROQ`)	APPROVED & MERGED (PR #55, #56, #57)